GitHub 인기 레포지토리 & AI 개발 도구 완전 활용 가이드북

> **대상:** 대학생 개발자 (비전공자 포함, 풀스택 개발자 지망)
> **기준:** GitHub Stars 10,000+ 레포지토리 중심
> **도구:** Claude Code + GSTACK + Sequential Thinking + Plan 모드 활용
> **버전:** 4.0 (구조 개선 + 자동화 파이프라인)

목차

- GitHub 인기 레포지토리 & AI 개발 도구 완전 활용 가이드북
- 이 가이드북에 대하여
  - 메타 접근: 가이드북이 스스로를 증명합니다
- 가이드북 구조
  - 추천 읽기 순서
- 사용된 도구
  - Claude Code Plan 모드
  - Sequential Thinking
  - GSTACK (Garry Tan's Skill Pack)
- 빠른 시작
- Part 1: 개발 기초 다지기
- 학습 목표
- 카테고리 4: 비전공자를 위한 학습 가이드
- 4-1. CS 기초 다지기
  - ossu/computer-science (178K+ Stars)
  - kamranahmedse/developer-roadmap (310K+ Stars)
  - jwasham/coding-interview-university (310K+ Stars)
- 4-2. 코딩 입문 - 실전 프로젝트
  - freeCodeCamp/freeCodeCamp (410K+ Stars)
  - TheOdinProject/curriculum (10K+ Stars)
  - codecrafters-io/build-your-own-x (330K+ Stars)
- 4-3. 면접 준비
  - yangshun/tech-interview-handbook (122K+ Stars)
  - donnemartin/system-design-primer (290K+ Stars)
  - kdn251/interviews (63K+ Stars)
- 4-4. 데이터 분석 & 시각화
  - Python 데이터 분석 기초
- Part 2: 프레임워크 및 언어별 학습 레포지토리
- 학습 목표
- 2-1. Spring Boot 생태계
  - 2-1-1. spring-projects/spring-boot (76K+)

  - 2-1-2. eugenp/tutorials (37K+) — Baeldung 예제 모음
  - 2-1-3. macrozheng/mall (83K+) — 전자상거래 시스템
  - 2-1-4. iluwatar/java-design-patterns (90K+)
  - 2-1-5. gothinkster/realworld (80K+)
- 2-2. React 생태계
  - 2-2-1. facebook/react (244K+)
  - 2-2-2. typescript-cheatsheets/react (45K+)
  - 2-2-3. alan2207/bulletproof-react (29K+)
- 2-3. Next.js 생태계
  - 2-3-1. vercel/next.js (130K+)
  - 2-3-2. vercel/commerce (11K+) — 커머스 스타터
- 2-4. 이 파트 학습 체크리스트
- Part 3: 프로젝트 및 공모전 활용
- 학습 목표
- 3-1. 풀스택 프로젝트 템플릿
  - 3-1-1. RealWorld (gothinkster/realworld)
  - 3-1-2. Full Stack FastAPI Template
  - 3-1-3. Redoc: API 문서화 도구
- 3-2. 포트폴리오 & 오픈소스 기여
  - 3-2-1. First Contributions
  - 3-2-2. Awesome for Beginners
  - 3-2-3. GitHub Opensource Guide
- 3-3. 해커톤 & 공모전
  - 3-3-1. Hackathon Starter
  - 3-3-2. AutoGPT
  - 3-3-3. LangChain
- Part 4: 문서 처리 라이브러리 완전 가이드
- 학습 목표
- 4-1. Python 문서 라이브러리
  - 4-1-1. python-docx (Word 문서)
  - 4-1-2. openpyxl (Excel 처리)
  - 4-1-3. python-pptx (PowerPoint)
  - 4-1-4. PyMuPDF (PDF 처리)
- 4-2. Java 문서 라이브러리
  - 4-2-1. Apache POI (Excel/Word/PowerPoint)
  - 4-2-2. iText (PDF 생성/편집)

- 4-3. JavaScript 문서 라이브러리
  - 4-3-1. SheetJS (xlsx)
  - 4-3-2. PDFKit
  - 4-3-3. docx (Word 생성)
- 4-4. 문서 자동화 파이프라인
  - Claude Code와 문서 라이브러리 통합
- 실전 활용: 포트폴리오 자동 생성
- Part 5: AI 개발 도구 — Claude Code MCP & GSTACK
- 학습 목표
- 5-1. 핵심 MCP 서버
  - MCP 소개
  - GitHub MCP - 레포지토리 관리
  - Filesystem MCP - 로컬 파일 시스템
  - PostgreSQL MCP - 데이터베이스 연동
  - Sequential Thinking MCP - 복잡한 분석
- 5-2. 생산성 MCP 서버
  - Playwright MCP - 브라우저 자동화
  - Memory MCP - 지속적인 지식 관리
  - Docker MCP - 컨테이너 관리
- 5-3. MCP 서버 조합 전략
  - 개발 워크플로우 (완전한 예제)
  - 문서 작업 워크플로우
- 5-4. GSTACK 설치 및 설정
  - 설치 방법
  - 23개 GSTACK 스킬 설명
- 5-5. GSTACK 역할별 활용 가이드
  - CEO 역할: 전략 수립
  - Engineer 역할: 기술 구현
  - QA 역할: 품질 보증
  - Release 역할: 배포 관리
- 5-6. GSTACK + MCP 시너지 워크플로우
  - 완전한 프로젝트 라이프사이클
- 5-7. Plan 모드 & Sequential Thinking 활용법
  - Plan 모드란?
  - Plan 모드 사용법
  - Sequential Thinking 활용법

  - 프로젝트 계획 통합 워크플로우
- 5-8. 실전 통합 프로젝트 사례
  - 사례: "AI 코드 리뷰 봇" SaaS 개발
- 5-9. GSTACK 팁 및 베스트 프랙티스
  - 효과적인 사용법
- Part 6: 취업 준비 및 면접 전략
- 학습 목표
- 6-1. 기술 면접 준비
  - yangshun/tech-interview-handbook (122K+ Stars)
  - donnemartin/system-design-primer (290K+ Stars)
  - kdn251/interviews (63K+ Stars)
- 6-2. 금융/공기업 취업에 도움되는 레포지토리
  - B-1. 금융 IT 관련 레포지토리
  - B-2. 공기업 코딩테스트 준비
  - B-3. 보안 & 인프라 (금융권 필수)
  - B-4. DevOps & CI/CD
- 부록
- 부록 A: Claude Code Plan 모드 완전 가이드
  - A-1. Plan 모드란?
  - A-2. Plan 모드 활성화 방법
  - A-3. Plan 모드 vs Direct 모드 사용 시기
  - A-4. Plan 모드 단계별 워크플로우
  - A-5. Plan 편집 (Ctrl+G)
  - A-6. Plan 모드 베스트 프랙티스
- 부록 B: Sequential Thinking 완전 가이드
  - B-1. Sequential Thinking MCP란?
  - B-2. Sequential Thinking 설치
  - B-3. Sequential Thinking 작동 원리
  - B-4. Sequential Thinking 사용 시기
  - B-5. Sequential Thinking 실제 예제
  - 파이프라인 구조
  - 사용된 라이브러리

이 가이드북에 대하여

이 가이드북은 **대학생 개발자 지망생**을 위한 실전 GitHub 레포지토리 활용서입니다.

단순히 "이런 레포가 있다"를 넘어, **각 레포지토리를 어떻게 설치하고, 어떻게 사용하고, 실전에서 어떻게 활용하는지**를 코드 예제와 함께 상세히 다룹니다.

메타 접근: 가이드북이 스스로를 증명합니다

이 가이드북의 특별한 점은 **Part 4에서 소개하는 문서 처리 라이브러리들로 이 가이드북 자체가 자동 생성된다**는 것입니다.

```
src/*.md  →  md_merger.py  →  output/Guidebook.md
                                      ↓
                              ┌──────────────┐
                              ↓              ↓
                        md_to_docx.py   md_to_pdf.py
                              ↓              ↓
                   output/Guidebook.docx  output/Guidebook.pdf
```

- **python-docx** (Part 4-1)로 Word 문서를 자동 생성
- **WeasyPrint** (Part 4-4 확장)로 PDF를 자동 생성
- **Pygments**로 코드 구문 강조를 적용

즉, "이 라이브러리로 이런 걸 만들 수 있다"를 **이 가이드북 자체가 증명**합니다.

가이드북 구조

파트	제목	대상	핵심 내용
Part 1	개발 기초 다지기	비전공자/입문자	CS 기초, 코딩 입문, 데이터 분석
Part 2	프레임워크/언어별 학습	초급~중급	Spring Boot, React, Next.js, Django 등
Part 3	프로젝트 및 공모전	중급	풀스택 프로젝트, 포트폴리오, 오픈소스 기여
Part 4	문서 처리 라이브러리	중급~고급	python-docx, openpyxl, WeasyPrint 등
Part 5	AI 개발 도구	모든 레벨	Claude Code MCP, GSTACK 워크플로우
Part 6	취업 준비/면접	취준생	기술 면접, 금융/공기업 전략
부록	참고 자료	모든 레벨	로드맵, 리소스, 빌드 가이드

추천 읽기 순서

완전 초보자: Part 1 → Part 2 → Part 3 → Part 6

프로그래밍 경험자: Part 2 → Part 3 → Part 4 → Part 5

취업 준비생: Part 6 → Part 3 → Part 4

AI 도구에 관심: Part 5 → Part 4 → Part 2

사용된 도구

Claude Code Plan 모드

이 가이드북의 구조 설계에 Plan 모드를 활용했습니다. 변경사항을 미리 검토하고 승인 후 적용하는 워크플로우로, 대규모 프로젝트에서 실수를 방지합니다.

Sequential Thinking

각 레포지토리 분석을 단계별로 수행했습니다:
1. 레포지토리 개요 파악
2. 설치 및 환경 설정
3. 기본 사용법 코드 작성
4. 실전 활용 시나리오 설계
5. 관련 레포지토리 연결

GSTACK (Garry Tan's Skill Pack)

가이드북 제작 전 과정에서 GSTACK 스킬을 활용했습니다:
- `/office-hours` → 전체 구조 전략 수립
- `/plan-ceo-review` → 파트별 가치 평가
- `/plan-eng-review` → 파이프라인 아키텍처 리뷰
- `/qa` → 콘텐츠 품질 검증
- `/ship` → 최종 산출물 배포

빠른 시작

```
# 이 가이드북을 직접 빌드해보세요!
git clone <repository-url>
cd PLAF

# 의존성 설치
pip install -r scripts/requirements.txt

# 가이드북 빌드 (MD + DOCX + PDF)
python scripts/build_guidebook.py
```

산출물은 `output/` 디렉토리에 생성됩니다.

Part 1: 개발 기초 다지기

> 이 파트는 프로그래밍을 처음 시작하는 분들을 위한 기초 가이드입니다.
> CS 기초부터 코딩 입문, 데이터 분석까지 단계별로 학습할 수 있습니다.

학습 목표

- 컴퓨터 과학 기본 개념 이해
- 프로그래밍 언어 기초 습득
- 개발 환경 설정 및 Git 사용법
- 데이터 분석 기초

카테고리 4: 비전공자를 위한 학습 가이드

개발자 지망생 중 컴퓨터 과학을 전공하지 않은 사람들을 위한 체계적인 학습 경로입니다.
이 카테고리는 기초부터 실전까지 단계적으로 성장할 수 있도록 구성했습니다.

4-1. CS 기초 다지기

ossu/computer-science (178K+ Stars)

링크: https://github.com/ossu/computer-science

전 세계 비전공자 학습자를 위한 완전한 컴퓨터 과학 커리큘럼입니다. 무료이며 대학 수준의 교육을 제공합니다.

📋 개요

- **제공 형식**: 온라인 강좌 모음 + 프로젝트 목록
- **총 소요 시간**: 180주 (주당 30-40시간 투자 기준)
- **커버 범위**: Fundamentals → Core → Advanced → Specialization
- **언어**: 대부분 영어 강좌 (한국어 자막 일부)

🚀 시작하기

1. **Repository 구조 이해**

```
# OSSU/CS 저장소 클론
git clone https://github.com/ossu/computer-science.git
cd computer-science

# README.md에서 커리큘럼 전체 구조 확인
cat README.md | head -100
```

1. **추천 학습 단계** (비전공자 기준)

단계	예상 기간	핵심 과목	학습량
1단계: 기초	12주	Programming (Python), Math Foundations	낮음
2단계: 핵심	24주	Data Structures, Algorithms, Database	중간
3단계: 심화	16주	OS, Networks, Compilers	높음
4단계: 전공선택	12주	ML, Security, Distributed Systems 중 선택	매우높음

💻 실전 학습법

Step 1: 프로그래밍 기초 (MIT CS50)
- CS50x Weeks 6-8에서 Python 문법, 함수, 리스트/딕셔너리 등 기본기를 다집니다.
- 과제 제출 플랫폼(check50)으로 즉시 피드백을 받을 수 있어 독학에 적합합니다.

Step 2: 자료구조 이해 (스택, 큐, 링크드리스트)
- UC San Diego의 Data Structures 강좌에서 개념을 잡고, OSSU 리포의 과제 링크를 따라 직접 구현합니다.
- 개념 이해가 먼저입니다. 종이에 그려본 뒤 Python `list`, `collections.deque`로 구현하세요.

Step 3: 알고리즘 기초 (정렬/탐색)
- Princeton Algorithms Part I에서 버블/병합/퀵 정렬, 이진 탐색 등을 학습합니다.
- 구현은 이미 레포 안 `coursework/` 폴더에 연습 문제로 포함되어 있습니다.

> Claude Code 활용 팁
> `> "OSSU CS 커리큘럼에서 비전공자에게 가장 중요한 3개 강좌만 골라서 학습 계획을 짜줘"`
> `> "이 레포의 Core CS 섹션을 읽고 내 수준(Python 기초만 앎)에 맞는 시작점을 추천해줘"`

⏱️ 권장 학습 일정

주 1-4: Python 기초 (MIT CS50)
- 강좌: CS50x Weeks 6-8 (Python)
- 시간: 주 8-10시간
- 프로젝트: Hello World, 간단한 계산기, To-do list 프로그램

주 5-8: 자료구조 (UC San Diego DSA)
- 강좌: Algorithms and Data Structures
- 시간: 주 12-15시간
- 프로젝트: Stack/Queue 구현, LinkedList 만들기

주 9-12: 알고리즘 (Princeton Algorithms)
- 강좌: Algorithms Part I & II
- 시간: 주 15-20시간
- 프로젝트: 정렬 알고리즘 성능 비교, 탐색 알고리즘 구현

💡 비전공자 특화 팁

1. **코드 작성 전 개념 이해**: 강좌 영상을 처음부터 끝까지 보고 난 후 코드 작성
2. **손으로 그려보기**: 자료구조는 반드시 종이에 그리면서 학습
3. **작은 프로젝트부터**: 완벽하지 않아도 작동하는 코드부터 시작
4. **온라인 커뮤니티**: OSSU Discord 서버에 한국어 채널 있음

---

kamranahmedse/developer-roadmap (310K+ Stars)

링크: https://github.com/kamranahmedse/developer-roadmap

개발자 진로를 시각화한 로드맵 모음. 각 단계별로 어떤 기술을 배워야 하는지 명확하게 보여줍니다.

📊 로드맵 종류

```
roadmap/
├── Frontend Developer Roadmap
│   ├── 1단계: HTML, CSS, JavaScript 기초
│   ├── 2단계: 패키지 매니저 (npm), 버전 관리 (git)
│   ├── 3단계: CSS 프레임워크 (Tailwind, Bootstrap)
│   ├── 4단계: JavaScript 심화 (DOM, AJAX)
│   └── 5단계: 프론트엔드 프레임워크 (React, Vue, Angular)
├── Backend Developer Roadmap
│   ├── 1단계: 언어 선택 (Node.js, Python, Java)
│   ├── 2단계: 웹 프레임워크 (Express, Django, Spring)
│   ├── 3단계: 데이터베이스 (SQL, NoSQL)
│   ├── 4단계: API 설계 (REST, GraphQL)
│   └── 5단계: 배포 & 확장 (Docker, Kubernetes)
├── DevOps Roadmap
│   ├── Linux 기초
│   ├── 컨테이너 (Docker)
│   ├── 오케스트레이션 (Kubernetes)
│   ├── CI/CD (GitHub Actions, Jenkins)
│   └── 모니터링 (Prometheus, ELK)
└── Full Stack Developer Roadmap
    └── Frontend + Backend + DevOps 통합
```

🎯 로드맵 활용법

1단계: 현재 수준 파악

```
당신의 수준이 어디인가요?
- 초급 (0-6개월 경력): Beginner roadmap 선택
- 중급 (6-18개월 경력): Intermediate roadmap 선택
- 고급 (18개월 이상): Advanced roadmap 선택
```

2단계: 우선순위 정하기

```
Frontend 개발자가 되고 싶다면:
Essential (필수): HTML → CSS → JavaScript → React
Good to Have (선택): TypeScript → Testing → Performance
Advanced (심화): Custom Build Tools → Web Performance → PWA
```

3단계: 시간 할당 전략
- 비전공자는 주제당 40-60시간을 잡습니다. 주 20시간 기준 주제 하나에 2-3주가 적당합니다.
- Frontend 로드맵 예: HTML → CSS → JavaScript → DOM → React → Hooks → State Management 순으로 각 3주.
- 로드맵의 각 노드는 "Personal Recommendation / Alternative Option"으로 구분되어 있어 우선순위를 바로 볼 수 있습니다.

> Claude Code 활용 팁
> `> "developer-roadmap 레포의 Frontend 로드맵을 보고 주 20시간 학습 기준 6개월 계획표를 만들어줘"`

🔗 연관 리소스 링크

각 로드맵의 항목마다 실제 학습 자료 링크가 포함되어 있습니다:
- FreeCodeCamp 튜토리얼
- Udemy 강좌
- MDN 문서
- 공식 문서

---

jwasham/coding-interview-university (310K+ Stars)

링크: https://github.com/jwasham/coding-interview-university

Google과 같은 대기업에 합격하기 위한 완벽한 인터뷰 준비 가이드입니다.

📚 커버 범위

```
Interview Prep Curriculum
├── 자료구조 (10일)
│   ├── Array
│   ├── LinkedList
│   ├── Stack
│   ├── Queue
│   ├── Hash Table
│   ├── Binary Search Tree
│   ├── Heap
│   └── Graph
├── 알고리즘 (20일)
│   ├── DFS (Depth First Search)
│   ├── BFS (Breadth First Search)
│   ├── Dynamic Programming
│   ├── Bit Manipulation
│   └── Combinatorics
├── 시스템 디자인 (5일)
│   ├── Scalability
│   ├── Database Design
│   ├── Caching
│   └── Load Balancing
└── 행동 질문 (3일)
    ├── STAR 기법
    ├── 프로젝트 설명
    └── 성과 이야기
```

💻 알고리즘 인터뷰 대비 코드

LeetCode Easy: Two Sum 문제

```python
def two_sum(nums: list, target: int) -> list:
    """
    주어진 리스트에서 합이 target이 되는 두 수의 인덱스를 반환

    예: nums = [2, 7, 11, 15], target = 9
    반환: [0, 1] (2 + 7 = 9)
    """
    # 방법 1: Brute Force (O(n²))
    # for i in range(len(nums)):
    #     for j in range(i + 1, len(nums)):
    #         if nums[i] + nums[j] == target:
    #             return [i, j]

    # 방법 2: Hash Map (O(n)) - 권장
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# 테스트
assert two_sum([2, 7, 11, 15], 9) == [0, 1]
assert two_sum([3, 2, 4], 6) == [1, 2]
print("✓ Two Sum 통과")
```

LeetCode Medium: Binary Search Tree Validate

```python
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def is_valid_bst(root: Optional[TreeNode]) -> bool:
    """
    주어진 이진 트리가 유효한 이진 탐색 트리인지 확인

    이진 탐색 트리 규칙:
    - 왼쪽 서브트리의 모든 값 < 노드값
    - 오른쪽 서브트리의 모든 값 > 노드값
    """
    def validate(node, min_val=float('-inf'), max_val=float('inf')):
        if not node:
            return True

        if node.val <= min_val or node.val >= max_val:
            return False

        return (validate(node.left, min_val, node.val) and
                validate(node.right, node.val, max_val))

    return validate(root)

# 테스트 케이스
#     2
#    / \
#   1   3
root = TreeNode(2)
root.left = TreeNode(1)
root.right = TreeNode(3)
assert is_valid_bst(root) == True
print("✓ Valid BST 확인")

# 잘못된 트리
#     5
#    / \
#   1   4
#      / \
#     3   6
bad_root = TreeNode(5)
bad_root.left = TreeNode(1)
bad_root.right = TreeNode(4)
bad_root.right.left = TreeNode(3)
bad_root.right.right = TreeNode(6)
assert is_valid_bst(bad_root) == False
print("✓ Invalid BST 확인")
```

LeetCode Hard: Median of Two Sorted Arrays

```python
def find_median_sorted_arrays(nums1: list, nums2: list) -> float:
    """
    두 정렬된 배열의 중앙값 찾기 (O(log(min(m,n))) 필수)
    """
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    left, right = 0, m

    while left <= right:
        partition1 = (left + right) // 2
        partition2 = (m + n + 1) // 2 - partition1

        maxLeft1 = float('-inf') if partition1 == 0 else nums1[partition1 - 1]
        minRight1 = float('inf') if partition1 == m else nums1[partition1]

        maxLeft2 = float('-inf') if partition2 == 0 else nums2[partition2 - 1]
        minRight2 = float('inf') if partition2 == n else nums2[partition2]

        if maxLeft1 <= minRight1 and maxLeft2 <= minRight2:
            if (m + n) % 2 == 0:
                return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) / 2
            else:
                return max(maxLeft1, maxLeft2)
        elif maxLeft1 > minRight2:
            right = partition1 - 1
        else:
            left = partition1 + 1

    return -1

# 테스트
assert find_median_sorted_arrays([1, 3], [2]) == 2.0
assert find_median_sorted_arrays([1, 2], [3, 4]) == 2.5
print("✓ Median of Two Sorted Arrays 통과")
```

📅 8주 준비 스케줄

```
1주: 자료구조 심화 (Array, LinkedList, Stack, Queue)
  - 매일 1개 문제 풀기 (LeetCode Easy)
  - 시간: 1-2시간

2주: 자료구조 심화 (Tree, Graph, Hash Table)
  - 매일 1-2개 문제 (Easy + Medium)
  - 시간: 2-3시간

3주: 정렬/탐색 알고리즘
  - 병합 정렬, 퀵 정렬 구현
  - BFS, DFS 마스터
  - 매일 2개 문제 (Medium)

4주: 동적 프로그래밍
  - DP 개념 정리
  - 클래식 DP 문제 풀이
  - 매일 1-2개 문제 (Medium-Hard)

5주: 시스템 디자인 기초
  - Scalability 개념
  - Database 설계
  - 주당 2-3개 설계 질문 준비

6주: 행동 질문 준비
  - STAR 기법 연습
  - 이력서 정리
  - 모의 면접 1회

7주: 실제 기출 문제 연습
  - 회사별 기출 문제 60문제
  - 매일 2-3시간 문제 풀기

8주: 최종 점검 및 모의 면접
  - 주당 1-2회 모의 면접
  - 약한 부분 집중 복습
```

🎤 인터뷰 팁

기술 면접 Checklist

```python
def technical_interview_checklist():
    checklist = {
        "소통": [
            "문제를 충분히 이해했는지 확인",
            "접근 방법을 설명하기",
            "트레이드오프 논의하기"
        ],
        "코딩": [
            "오류 처리 고려",
            "엣지 케이스 처리",
            "코드 정리하기"
        ],
        "최적화": [
            "시간복잡도 분석",
            "공간복잡도 분석",
            "더 나은 솔루션 제시"
        ],
        "테스트": [
            "샘플 입력으로 테스트",
            "엣지 케이스 테스트",
            "성능 검증"
        ]
    }

    for category, items in checklist.items():
        print(f"\n[{category}]")
        for item in items:
            print(f"  □ {item}")

technical_interview_checklist()
```

4-2. 코딩 입문 - 실전 프로젝트

freeCodeCamp/freeCodeCamp (410K+ Stars)

링크: https://github.com/freeCodeCamp/freeCodeCamp

수백 시간의 무료 코딩 교육. 초급부터 고급까지 단계별로 학습할 수 있습니다.

🎓 커리큘럼 구조

```
freeCodeCamp Curriculum
├── Responsive Web Design (300시간)
│   ├── HTML & CSS 기초
│   ├── Flexbox & Grid
│   └── 5개 프로젝트 (Portfolio, Survey, Landing Page 등)
├── JavaScript Algorithms and Data Structures (300시간)
│   ├── JavaScript 기초
│   ├── ES6+ 문법
│   ├── 자료구조 & 알고리즘
│   └── 5개 프로젝트
├── Front End Development Libraries (300시간)
│   ├── Bootstrap
│   ├── jQuery
│   ├── React
│   ├── Redux
│   └── 5개 프로젝트
├── Data Visualization (300시간)
│   ├── D3.js
│   ├── API & Fetch
│   └── 4개 프로젝트
└── Back End Development and APIs (300시간)
    ├── Node.js & Express
    ├── MongoDB
    ├── User Authentication
    └── 5개 프로젝트
```

💻 시작 가이드

Step 1: 개발 환경 설정

```
# Node.js 설치 확인
node --version
npm --version

# VS Code 설치
# https://code.visualstudio.com/

# 권장 확장 프로그램
# - Live Server
# - Thunder Client (API 테스트)
# - Code Runner
# - Prettier (자동 포맷팅)
```

Step 2: 첫 번째 프로젝트 - Personal Portfolio

- freeCodeCamp Responsive Web Design 섹션에서 HTML/CSS Flexbox·Grid를 배운 뒤 포트폴리오 페이지를 만듭니다.
- 필요한 섹션: Navbar, Hero, About, Projects Grid, Contact. 반응형(@media 768px 기준)은 필수.
- Smooth scroll, sticky navbar 등 JS 상호작용은 작은 스니펫만 추가하면 충분합니다.

> Claude Code 활용 팁
> `> "freeCodeCamp 커리큘럼의 Responsive Web Design 프로젝트 5개를 단계별로 어떻게 풀어가는지 알려줘"`
> `> "내가 만든 index.html/style.css를 리뷰하고 접근성(a11y)·반응형 관점에서 개선점을 알려줘"`

TheOdinProject/curriculum (10K+ Stars)

링크: https://github.com/TheOdinProject/curriculum

완전한 오픈소스 풀스택 웹개발 커리큘럼. HTML부터 배포까지 모든 것을 배웁니다.

🗺️ 학습 경로

```
The Odin Project Path
├── Foundations (10-20주)
│   ├── Installing Tools
│   ├── How This Course Will Work
│   ├── How the Web Works
│   ├── JavaScript Basics
│   ├── HTML Basics
│   ├── CSS Basics
│   ├── Flexbox
│   ├── Grid
│   └── 10개 프로젝트
├── Intermediate HTML and CSS (4-8주)
│   ├── Intermediate HTML Concepts
│   ├── Intermediate CSS Concepts
│   ├── 2개 프로젝트
├── JavaScript (12-20주)
│   ├── Fundamentals
│   ├── DOM Manipulation
│   ├── OOP
│   ├── Async JavaScript
│   └── 10개 프로젝트
├── Getting Hired (진행 중 진행)
│   ├── Portfolio Project
│   ├── Preparing for Interviews
│   ├── Getting Hired
└── Paths (경로 선택)
    ├── Foundations → React (Front-end)
    │   ├── React Basics
    │   ├── Advanced React
    │   ├── 5개 프로젝트
    │   └── Capstone Project
    └── Foundations → Rails (Full Stack)
        ├── Ruby Basics
        ├── Ruby on Rails
        ├── Databases
        ├── 5개 프로젝트
        └── Capstone Project
```

📌 주요 프로젝트

1. 가위바위보 게임
- The Odin Project Foundations 과정의 첫 JavaScript 프로젝트. 3개 버튼 + 랜덤 선택 + 승패 로직만으로 완성됩니다.
- 학습 포인트: `Math.random()`, 조건문, DOM 이벤트 핸들러, 상태 관리.

2. 계산기 프로젝트 (고급)
- ES6 Class 문법, 상태 관리(`currentOperand`, `previousOperand`), 데이터 속성(`[data-number]`) 활용을 배웁니다.
- 단순히 만들지 말고 "소수점 2번 입력 방지", "0으로 나누기 예외" 같은 엣지 케이스도 직접 처리하면 실력이 붙습니다.

> Claude Code 활용 팁
> `> "The Odin Project의 Calculator 프로젝트 요구사항을 보고 내가 놓친 기능이 있는지 체크해줘"`

---

codecrafters-io/build-your-own-x (330K+ Stars)

링크: https://github.com/codecrafters-io/build-your-own-x

유명한 도구들을 처음부터 만들어보는 프로젝트 모음입니다.

🏗 구성 요소별 학습

```
Build Your Own X
├── Build Your Own Git (Rust)
│   ├── 1. Read Objects
│   ├── 2. Reading Commits
│   ├── 3. Read Tree Objects
│   ├── 4. Writing Objects
│   └── 5. Creating Commits
├── Build Your Own Docker (Go)
│   ├── 1. Container Basics
│   ├── 2. Namespaces
│   ├── 3. Cgroups
│   └── 4. Docker CLI
├── Build Your Own Redis (Python/Ruby/Go)
│   ├── 1. TCP Server
│   ├── 2. RESP Protocol
│   ├── 3. Expiration
│   └── 4. Replication
└── Build Your Own Language (Go/Python)
    ├── 1. Lexer
    ├── 2. Parser
    ├── 3. Evaluator
    └── 4. Advanced Features
```

💾 예시: Redis 만들기 (Python)

- 레포 안 `Build Your Own Redis` 튜토리얼이 단계별 학습 과제를 제공합니다.
- 핵심 학습 순서: TCP 소켓 서버 → RESP 프로토콜 파싱 → SET/GET/DEL 명령어 → 만료(EX 옵션) → 복제.
- Python `socket`, `threading`, `datetime` 세 개 모듈만으로 최소 구현이 가능합니다.

> Claude Code 활용 팁
>
> `> "build-your-own-x 레포에서 Build Your Own Redis 챕터를 읽고, 비전공자가 1주차에 구현해볼 수 있는 가장 작은 서브셋을 골라줘"`

📚 학습 로드맵

```
초급 (1개월)
- Git 기본 개념 이해
- 간단한 TCP 서버 만들기
- 기본 프로토콜 파싱

중급 (2-3개월)
- Redis 서버 전체 구현
- Docker 컨테이너 기초
- 동시성 처리

고급 (3-6개월)
- 분산 시스템 개념
- 프로그래밍 언어 만들기
- 성능 최적화
```

4-3. 면접 준비

yangshun/tech-interview-handbook (122K+ Stars)

링크: https://github.com/yangshun/tech-interview-handbook

개발자 면접을 위한 종합 가이드. 구성, 질문, 답변 전략까지 담고 있습니다.

📋 면접 유형별 전략

1. 코딩 면접 (Coding Interview)

```python
# 면접 전 체크리스트
interview_checklist = {
    "인터뷰 전": [
        "회사 기술 스택 조사",
        "최근 프로젝트 복습",
        "인터뷰어 LinkedIn 확인",
        "자주 묻는 문제 3-5개 준비"
    ],
    "인터뷰 중": [
        "질문 명확히 이해하기",
        "접근 방법 설명하기",
        "엣지 케이스 고려하기",
        "시간복잡도 분석하기",
        "트레이드오프 논의하기"
    ],
    "코딩 후": [
        "코드 리뷰하기",
        "테스트 케이스 작성하기",
        "성능 개선 제시하기",
        "결과 검증하기"
    ]
}

for phase, items in interview_checklist.items():
    print(f"\n📌 {phase}")
    for item in items:
        print(f"  □ {item}")
```

2. 시스템 디자인 면접

```
면접 프레임워크:

1. 요구사항 명확히 하기 (5분)
   - 함수형 요구사항 (FRs)
   - 비함수형 요구사항 (NFRs)
   - Scale, QPS, Data volume

2. API 설계 (5분)
   - Endpoints 정의
   - Request/Response 형식
   - Rate limiting

3. 데이터 모델 설계 (5분)
   - Database schema
   - Indexing
   - Sharding strategy

4. 높은 수준의 설계 (10분)
   - Components
   - Data flow
   - Caching layer
   - Message queue

5. 상세 설계 (10분)
   - Core components 심화
   - Bottleneck 분석
   - Optimization

6. 마무리 (5분)
   - Monitoring
   - Logging
   - Future improvements
```

실제 시스템 디자인 예: URL Shortener

```
1단계: 요구사항
- 함수형: 긴 URL → 짧은 URL 변환, 리다이렉트
- 비함수형:
  * 초당 쓰기: 500 writes/sec
  * 초당 읽기: 100,000 reads/sec
  * 가용성: 99.9%
  * Latency: < 200ms

2단계: 용량 산정
- 5년 데이터: 500 * 86400 * 365 * 5 = 78.8 billion URLs
- 저장 공간: 78.8B * 100 bytes = 7.88 TB

3단계: API 설계
POST /api/shorten
{
    "long_url": "https://example.com/very/long/url"
}
응답:
{
    "short_url": "https://short.url/abc123"
}

GET /api/{short_code}
응답: 301 Redirect to original URL

4단계: 데이터 모델
Table: URLMapping
- id (PK): bigint
- short_code: varchar(10) UNIQUE
- long_url: varchar(2048)
- created_at: timestamp
- expiration: timestamp

5단계: 높은 수준의 설계
Client → Load Balancer → API Servers
                            ↓
                       Cache (Redis)
                            ↓
                     Database (MySQL)
                            ↓
                     Message Queue (Kafka)
                            ↓
                     Analytics Service

6단계: 상세 설계
- Short code 생성: Base62 encoding (0-9, a-z, A-Z)
- Collision 처리: Retry with increment
- Caching 전략: LRU cache in Redis
- Load 분산: Consistent hashing
```

---

donnemartin/system-design-primer (290K+ Stars)

링크: https://github.com/donnemartin/system-design-primer

확장 가능한 시스템 설계를 위한 종합 리소스입니다.

🏗 핵심 개념

1. Scalability (확장성)

```
Vertical Scaling (수직 확장)
- CPU 업그레이드
- 메모리 추가
- 한계: 단일 기계의 물리적 제약

Horizontal Scaling (수평 확장)
- 더 많은 서버 추가
- Load balancer로 트래픽 분산
- 권장: 대부분의 경우 수평 확장이 더 효율적

예시: 100 QPS → 1000 QPS로 확장
Before (단일 서버):
  [Application] → [Database] → Bottleneck!

After (수평 확장):
  Load Balancer
      ↓
  App1  App2  App3
      ↓   ↓   ↓
    Cache (Redis)
      ↓
    Database (Master-Slave)
      ↓
  Slave1  Slave2
```

2. 데이터베이스 설계

```
SQL vs NoSQL 선택

SQL (관계형 데이터베이스)
장점:
- ACID 보장
- 복잡한 쿼리 가능
- 데이터 무결성
단점:
- 수평 확장 어려움
- 고정된 스키마

예: MySQL, PostgreSQL

NoSQL (비관계형 데이터베이스)
장점:
- 수평 확장 용이
- 빠른 읽기/쓰기
- 유연한 스키마
단점:
- ACID 보장 안 함
- 쿼리 능력 제한
- Consistency 문제

예: MongoDB, Cassandra, DynamoDB
```

3. 캐싱 전략
- LRU(Least Recently Used), LFU(Least Frequently Used), Write-Through/Write-Back 등 전략별 차이를 이해합니다.
- Python에서는 `collections.OrderedDict` 또는 `functools.lru_cache` 데코레이터로 쉽게 실험할 수 있습니다.
- system-design-primer는 실제 프로덕션 시스템(Memcached, Redis, CDN)에서 각 전략이 어떻게 쓰이는지 사례와 함께 설명합니다.

---

kdn251/interviews (63K+ Stars)

링크: https://github.com/kdn251/interviews

Java 기반 면접 준비용 알고리즘 및 자료구조 구현.

🎯 Java 기반 면접 준비

- 이 레포는 LeetCode/HackerRank 빈출 문제의 Java 풀이를 카테고리별(Array, String, Tree, DP 등)로 정리해두었습니다.
- 활용법: 문제를 먼저 직접 풀어본 뒤, 이 레포의 풀이와 시간/공간 복잡도를 비교하는 식으로 학습합니다.
- Two Sum, Valid Palindrome, Reverse Integer 같은 Easy 문제로 시작해서 점차 Medium/Hard로 올라가세요.

> Claude Code 활용 팁
> `> "kdn251/interviews 레포의 Array 섹션에서 대기업 면접 빈출 Top 10을 골라줘"`

4-4. 데이터 분석 & 시각화

Python 데이터 분석 기초

설치 및 환경 설정

```
# 가상 환경 생성
python -m venv data-env
source data-env/bin/activate  # Linux/Mac
data-env\Scripts\activate  # Windows

# 필수 라이브러리 설치
pip install jupyter numpy pandas matplotlib seaborn scikit-learn streamlit
```

📊 Pandas를 이용한 데이터 분석

핵심 API 5가지만 알면 80%의 분석이 가능합니다.

```
import pandas as pd

df = pd.read_csv('data.csv')
df.head(); df.info(); df.describe()          # 탐색
df.fillna(df.mean(numeric_only=True))        # 결측 처리
df.drop_duplicates()                         # 중복 제거
df.groupby('category')['sales'].sum()        # 그룹 집계
df.corr(numeric_only=True)                   # 상관관계
```

시각화는 `matplotlib` / `seaborn` 기본 차트(hist, boxplot, scatter, heatmap)로 충분하고, 머신러닝은 `scikit-learn`의 `train_test_split` → `StandardScaler` → `RandomForestClassifier` 3단계 파이프라인을 먼저 익히세요.

📊 Streamlit으로 대시보드 만들기

- Streamlit은 Python 스크립트 한 개로 웹 대시보드를 만드는 도구입니다. HTML/CSS/JS 몰라도 됩니다.
- 핵심 API: `st.title`, `st.sidebar`, `st.metric`, `st.line_chart`, `st.bar_chart`, `st.dataframe`, `@st.cache_data`.
- 실행: `streamlit run dashboard.py`로 로컬 서버가 뜹니다.

> Claude Code 활용 팁
> `> "내 CSV 파일(컬럼: date, category, sales)로 Streamlit 대시보드 초안을 만들어줘. KPI 4개 + 시계열 차트 + 카테고리 필터 포함해서."`

```
# Streamlit 실행
streamlit run dashboard.py
```

Part 2: 프레임워크 및 언어별 학습 레포지토리

> 실전 개발에 필요한 주요 프레임워크와 언어별 학습 레포지토리입니다.
> Spring Boot, React, Next.js 등을 체계적으로 학습할 수 있습니다.

학습 목표

- 백엔드 프레임워크 (Spring Boot) 활용법 이해
- 프론트엔드 프레임워크 (React, Next.js) 학습 경로 파악
- 각 레포지토리에서 "어디를 먼저 읽어야 하는지" 파악
- Claude Code로 대형 오픈소스 코드베이스를 탐색하는 방법 체득

이 파트는 10,000+ Stars를 보유한 주요 웹 개발 프레임워크 레포지토리를 **"어떻게 클론해서, 어디를 보고, 무엇을 배울지"** 중심으로 안내합니다. 전체 코드를 옮겨 적는 대신 **레포 구조 · 핵심 파일 위치 · Claude Code 질문 예시**를 제공합니다.

2-1. Spring Boot 생태계

2-1-1. spring-projects/spring-boot (76K+)

레포지토리 개요

Spring Boot는 Spring Framework 기반의 빠른 애플리케이션 개발(RAD) 프레임워크입니다. 자동 설정(Auto-Configuration)과 내장 서버로 선언적 설정을 최소화하고 프로덕션 환경 배포를 단순화합니다. Java 백엔드 개발의 사실상 표준이며 마이크로서비스 아키텍처의 기반이 됩니다.

- Stars: 76,000+
- Language: Java
- License: Apache 2.0
- 공식 사이트: https://spring.io/projects/spring-boot

클론 및 사전 준비

```
# JDK 17+ 설치 확인
java -version

# 레포지토리 클론 (크기가 크므로 shallow clone 권장)
git clone --depth 1 https://github.com/spring-projects/spring-boot.git
cd spring-boot
```

디렉토리 구조에서 주목할 곳

- `spring-boot-project/spring-boot` — 코어 모듈, `@SpringBootApplication` 구현
- `spring-boot-project/spring-boot-autoconfigure` — 자동 설정 핵심 로직
- `spring-boot-project/spring-boot-starters` — 스타터 의존성 모음
- `spring-boot-project/spring-boot-docs` — 공식 레퍼런스 문서 소스
- `spring-boot-tests/spring-boot-smoke-tests` — 작은 샘플 앱 모음(학습에 최적)

학습 루트: 본 레포 대신 start.spring.io 사용

실제 신규 프로젝트는 https://start.spring.io/ 에서 생성하고, spring-boot 레포는 **내부 동작을 이해할 때만** 탐색하는 것이 좋습니다.

```
# Spring Initializr CLI (선택)
curl https://start.spring.io/starter.zip \
  -d dependencies=web,data-jpa,h2 \
  -d name=demo -o demo.zip
```

Claude Code 연동 팁

대형 레포는 한 번에 읽을 수 없습니다. 다음과 같이 질문하세요.

> "spring-boot-project/spring-boot-autoconfigure 디렉토리에서 DataSourceAutoConfiguration이 어떻게 조건부로 로드되는지 설명해 줘."
>
> "@SpringBootApplication 이 내부적으로 어떤 어노테이션을 조합하는지 소스에서 찾아 줘."
>
> "spring-boot-smoke-tests 중 REST API 가장 기본 예제를 하나 골라 구조를 설명해 줘."

2-1-2. eugenp/tutorials (37K+) — Baeldung 예제 모음

레포지토리 개요

Baeldung 블로그의 모든 예제 코드가 모듈별로 정리된 Java/Spring 학습 보물창고입니다. Spring Security, JPA, Kafka, Reactive, JWT 등 거의 모든 주제의 **작동하는 최소 예제**를 제공합니다.

- Stars: 37,000+
- 구조: Maven 멀티 모듈 (수백 개)
- 공식 블로그: https://www.baeldung.com/

클론 후 필요한 모듈만 빌드

```
git clone --depth 1 https://github.com/eugenp/tutorials.git
cd tutorials

# 전체 빌드는 30분 이상 걸립니다. 필요한 모듈만 빌드하세요.
cd spring-security-modules/spring-security-web-login
mvn spring-boot:run
```

학습자가 먼저 봐야 할 모듈

- `spring-boot-modules/` — Spring Boot 기초
- `spring-security-modules/` — 인증/인가 (JWT, OAuth2 포함)
- `persistence-modules/spring-data-jpa-*` — JPA, Query 메서드
- `testing-modules/` — JUnit 5, Mockito, Testcontainers

Claude Code 질문 예시

> "spring-security-modules 디렉토리에서 JWT 기반 로그인을 구현한 가장 단순한 모듈을 찾아 README와 주요 설정 파일만 요약해 줘."
>
> "persistence-modules 안에서 Spring Data JPA `@Query` 어노테이션 사용 예제를 3개 찾아 비교해 줘."

2-1-3. macrozheng/mall (83K+) — 전자상거래 시스템

레포지토리 개요

실제 운영 수준의 **쇼핑몰 백엔드 풀스택 예제**입니다. Spring Boot + MyBatis + Elasticsearch + Redis + RabbitMQ + MongoDB 조합을 실제 비즈니스 요구사항에 맞게 통합한 대형 프로젝트입니다.

- Stars: 83,000+
- 모듈: mall-admin(관리자), mall-portal(사용자), mall-search(검색), mall-demo

클론 및 실행 준비

```
git clone --depth 1 https://github.com/macrozheng/mall.git
cd mall

# 필수 인프라: MySQL 5.7+, Redis, (선택)Elasticsearch/RabbitMQ/MongoDB
# Docker Compose 사용을 강력 권장
# document/docker/ 디렉토리에 구성 파일 있음
```

디렉토리 구조에서 주목할 곳

- `mall-admin/` — 관리자 백오피스 API
- `mall-portal/` — 사용자 쇼핑몰 API (주문, 장바구니, 결제)
- `mall-search/` — Elasticsearch 기반 상품 검색
- `document/` — 아키텍처 다이어그램 · DB 스키마 · API 문서 (**반드시 먼저 읽기**)

Claude Code 학습 시나리오

> "mall-portal 모듈에서 주문 생성(Order)부터 결제 완료까지 흐름을 Service 계층 위주로 추적해 줘."
>
> "mall 프로젝트의 Redis 캐싱 전략이 어떤 Service 메서드에 적용됐는지 목록화해 줘."
>
> "document 디렉토리의 DB 스키마를 보고 상품-카테고리-SKU 관계를 ER 다이어그램으로 설명해 줘."

2-1-4. iluwatar/java-design-patterns (90K+)

레포지토리 개요

GoF 디자인 패턴 + 엔터프라이즈 패턴을 Java로 구현한 세계 최대 패턴 모음입니다. 각 패턴마다 독립 Maven 모듈, 클래스 다이어그램, README가 준비되어 있어 **한 패턴당 30분**이면 학습할 수 있습니다.

- Stars: 90,000+
- 패턴 수: 150+ (Creational, Structural, Behavioral, Concurrency, Architectural 등)

클론 및 특정 패턴 실행

```
git clone --depth 1 https://github.com/iluwatar/java-design-patterns.git
cd java-design-patterns

# 원하는 패턴 디렉토리로 이동 후 실행
cd singleton
mvn compile exec:java
```

초보자가 먼저 봐야 할 패턴

- `singleton/`, `factory/`, `builder/` — Creational 기초
- `strategy/`, `observer/`, `command/` — Behavioral 필수
- `adapter/`, `decorator/`, `facade/` — Structural 필수
- `repository/`, `dao/` — 백엔드 실전에서 자주 쓰임

Claude Code 질문 예시

> "이 레포의 strategy 패턴 README와 App.java를 읽고, 실제 결제 시스템(카드/카카오페이/토스)에 어떻게 적용할지 구조만 제안해 줘."
>
> "decorator 패턴과 chain-of-responsibility 패턴의 차이를 이 레포 예제를 인용해 설명해 줘."

2-1-5. gothinkster/realworld (80K+)

레포지토리 개요

"모든 프레임워크로 동일한 스펙의 블로그 앱(Conduit)을 구현한다"는 컨셉의 메타 레포지토리입니다. **프론트엔드 · 백엔드 100+ 구현체**가 모두 동일한 API 스펙을 따르므로, 같은 기능을 여러 기술 스택으로 비교 학습할 수 있습니다.

- Stars: 80,000+
- API 스펙: https://docs.realworld.show/

활용 방법

```
# 메인 레포 클론 (스펙과 구현체 링크만 있음)
git clone --depth 1 https://github.com/gothinkster/realworld.git

# 원하는 구현체는 별도 레포로 존재합니다. 예: Spring Boot 구현
git clone https://github.com/gothinkster/spring-boot-realworld-example-app.git
```

학습 포인트

- 동일 기능을 React / Vue / Angular / Svelte로 비교
- 동일 API를 Spring Boot / Express / Django / Rails / NestJS로 비교
- 자기 스택의 구현체를 읽고 **다른 스택의 구현체로 포팅**해 보는 연습

Claude Code 질문 예시

> "Spring Boot 구현체(gothinkster/spring-boot-realworld-example-app)와 Express 구현체(gothinkster/node-express-realworld-example-app)에서 Article 생성 API 비즈니스 로직을 비교해 줘."

2-2. React 생태계

2-2-1. facebook/react (244K+)

레포지토리 개요

React 자체의 소스 코드 레포지토리입니다. 신규 프로젝트는 이 레포에서 시작하지 않고, **Vite / Next.js / Create React App** 등 도구로 생성합니다. 본 레포는 **React 내부 동작을 이해**하고 싶을 때 탐색합니다.

- Stars: 244,000+
- License: MIT
- 공식 문서: https://react.dev/

신규 프로젝트 생성 (권장 경로)

```
# Vite 사용 (가장 빠름)
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev
```

레포에서 주목할 곳

- `packages/react` — 코어 라이브러리, Hooks 구현
- `packages/react-dom` — DOM 렌더러
- `packages/react-reconciler` — Fiber 아키텍처
- `packages/scheduler` — 우선순위 기반 스케줄러

Claude Code 질문 예시

> "packages/react/src/ReactHooks.js 를 보고 useState가 내부적으로 어떤 구조로 저장되는지 설명해 줘."
>
> "Fiber 아키텍처가 왜 도입됐는지 packages/react-reconciler README와 핵심 파일을 근거로 요약해 줘."

2-2-2. typescript-cheatsheets/react (45K+)

레포지토리 개요

React + TypeScript 조합에서 자주 겪는 타입 문제를 커뮤니티가 정리한 치트시트 모음입니다. 실제 코드 레포라기보다는 **문서 레포**에 가깝습니다. 실무에서 타입 에러를 만났을 때 가장 먼저 검색할 곳입니다.

- Stars: 45,000+
- 구성: basic / advanced / migrating / hoc 치트시트

활용 방법

```
# 별도 클론 없이 웹에서 바로 읽기 가능
# https://react-typescript-cheatsheet.netlify.app/

git clone --depth 1 https://github.com/typescript-cheatsheets/react.git
```

먼저 읽어야 할 문서

- `README.md` — 전체 인덱스
- `docs/basic/` — Props / State / Event / Forms 타입 정의
- `docs/advanced/` — Generic 컴포넌트, Polymorphic 컴포넌트
- `docs/hoc/` — HOC와 Render Prop의 제네릭

Claude Code 질문 예시

> "이 레포 docs/basic 을 기반으로 Input 컴포넌트의 onChange 이벤트 타입을 정의하는 방법 3가지를 비교해 줘."
>
> "advanced 섹션에서 'Generic Components' 예제를 읽고 내 프로젝트의 List 컴포넌트에 어떻게 적용할지 제안해 줘."

2-2-3. alan2207/bulletproof-react (29K+)

레포지토리 개요

"확장 가능한 React 프로젝트 구조"의 모범 답안으로 자주 언급되는 레포입니다. feature 기반 폴더 구조, 상태 관리 분리, 테스트 전략, 배포 설정까지 실무 베스트 프랙티스를 하나의 앱으로 구현했습니다.

- Stars: 29,000+
- 스택: React + TypeScript + Vite + Zustand + React Query + MSW

클론 및 실행

```
git clone --depth 1 https://github.com/alan2207/bulletproof-react.git
cd bulletproof-react/apps/react-vite
npm install
npm run dev
```

디렉토리 구조의 핵심 (필독)

- `docs/` — 프로젝트 철학 문서 (**먼저 읽기**)
- `src/features/` — 기능별 모듈 (auth, comments, discussions, users 등)
- `src/components/` — 공용 UI 컴포넌트
- `src/lib/` — 외부 라이브러리 래퍼 (axios, react-query 설정)
- `src/testing/` — 테스트 유틸 + MSW 핸들러

Claude Code 학습 시나리오

> "docs 디렉토리의 project-structure.md 와 application-overview.md 를 읽고 이 프로젝트의 폴더 구조 규칙을 내 프로젝트에 적용할 수 있는 체크리스트로 만들어 줘."
>
> "src/features/discussions 디렉토리의 api, components, routes, types 가 어떻게 상호작용하는지 추적해 줘."

2-3. Next.js 생태계

2-3-1. vercel/next.js (130K+)

레포지토리 개요

React 기반 풀스택 프레임워크. App Router · Server Components · Server Actions · 이미지 최적화 · 파일 기반 라우팅 등 현대 웹 개발의 사실상 표준 기능을 제공합니다. 신규 프로젝트는 `create-next-app` 으로 생성하고, 본 레포는 **내부 동작 이해와 예제 탐색**에 사용합니다.

- Stars: 130,000+
- 공식 문서: https://nextjs.org/docs

신규 프로젝트 생성

```
npx create-next-app@latest my-app --typescript --eslint --tailwind --app
cd my-app
npm run dev
# http://localhost:3000
```

Next.js 레포에서 주목할 곳

- `examples/` — **가장 중요**. 100+ 공식 예제(인증, DB, 결제, i18n 등)
- `packages/next` — 프레임워크 코어
- `docs/` — 공식 문서 소스
- `test/` — E2E 테스트 (실전 시나리오의 보고)

추천 예제 디렉토리 (examples/ 하위)

- `with-supabase` — 인증 + DB 풀스택
- `with-stripe-typescript` — 결제 연동
- `blog-starter` — 마크다운 블로그
- `with-next-auth` — 소셜 로그인
- `api-routes-rest` — REST API 패턴

Claude Code 질문 예시

> "vercel/next.js 레포의 examples/blog-starter 구조를 설명하고, 내 가이드북 프로젝트에 옮겨올 때 수정해야 할 부분을 알려 줘."
>
> "examples/with-next-auth 를 읽고 Google OAuth 로그인을 추가하는 최소 수정 단계를 정리해 줘."

> "App Router의 layout.tsx 와 Pages Router의 _app.tsx 차이를 공식 docs 디렉토리에서 근거를 찾아 비교해 줘."

2-3-2. vercel/commerce (11K+) — 커머스 스타터

레포지토리 개요

Next.js + Shopify 기반 **프로덕션급 커머스 템플릿**입니다. Server Components, Server Actions, 캐싱 전략 등 최신 Next.js 기능의 모범 사례를 담고 있어 **App Router 진지하게 배우고 싶다면 필독** 레포입니다.

활용 방법

```
git clone --depth 1 https://github.com/vercel/commerce.git
cd commerce
# README의 Shopify 설정 단계를 따라야 실행 가능
```

주목할 곳

- `app/` — App Router 구조, 라우트 그룹, 병렬/인터셉트 라우트
- `components/` — 공용 UI + cart 컴포넌트
- `lib/shopify/` — 외부 API 연동 패턴 (Server-only 함수)

Claude Code 질문 예시

> "vercel/commerce 의 app/product/[handle]/page.tsx 가 어떻게 Shopify 데이터를 Server Component 에서 페칭하는지 설명해 줘."
> "이 레포의 revalidateTag 사용 패턴을 내 블로그 프로젝트에 어떻게 적용할지 제안해 줘."

2-4. 이 파트 학습 체크리스트

- [ ] Spring Boot: `start.spring.io` 로 최소 프로젝트를 하나 만들어 실행했다
- [ ] eugenp/tutorials 에서 관심 있는 1개 모듈을 골라 README를 정독했다
- [ ] iluwatar/java-design-patterns 에서 3개 패턴을 직접 실행해 봤다
- [ ] Vite + React + TypeScript 프로젝트를 생성해 Hooks 예제를 작성했다
- [ ] bulletproof-react 의 `docs/` 디렉토리를 모두 읽었다
- [ ] `create-next-app` 으로 App Router 프로젝트를 생성해 1개 페이지를 만들었다

- [ ] vercel/next.js 의 `examples/` 에서 관심 있는 예제 1개를 클론해 실행했다
- [ ] 각 레포에 대해 Claude Code 에게 최소 1개 이상의 구조 질문을 해봤다

> 다음 파트: **Part 3 — 프로젝트 및 공모전 레포지토리**
> 학습한 프레임워크를 실전 포트폴리오로 연결하는 방법을 다룹니다.

Part 3: 프로젝트 및 공모전 활용

> 실전 프로젝트 경험을 쌓고, 포트폴리오를 만들고, 오픈소스에 기여하는 방법을 다룹니다.

학습 목표

- 풀스택 프로젝트 템플릿 활용
- 포트폴리오 구축 전략
- 오픈소스 기여 방법
- 해커톤/공모전 준비

3-1. 풀스택 프로젝트 템플릿

3-1-1. RealWorld (gothinkster/realworld)

GitHub Stars: 80K+
프로젝트 설명: Medium 클론 애플리케이션 - 모든 주요 프레임워크로 구현된 풀스택 프로젝트
용도: 풀스택 개발자 역량 증명용 포트폴리오 프로젝트

개요

RealWorld는 다양한 프레임워크(Spring Boot, React, Django, Node.js 등)로 동일한 애플리케이션을 구현한 메가 레포입니다. 블로그 플랫폼인 "Conduit"을 구현하며, 사용자 인증, 게시물 CRUD, 팔로우 기능, 즐겨찾기 등을 포함합니다.

설치 방법

```
# RealWorld 레포 클론
git clone https://github.com/gothinkster/realworld.git
cd realworld

# 원하는 백엔드 구현 선택 (예: Spring Boot)
cd implementations/spring-boot-gradle-jpa

# 의존성 설치
./gradlew build

# 데이터베이스 설정 (application.yml)
# spring.datasource.url: jdbc:h2:mem:testdb
# spring.jpa.hibernate.ddl-auto: create-drop
```

핵심 코드 구조

User 도메인 (Spring Boot)

```java
@Entity
@Getter
@Setter
@NoArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;

    @Column(unique = true)
    private String email;

    private String password;
    private String bio;
    private String image;

    @OneToMany(mappedBy = "author")
    private List<Article> articles = new ArrayList<>();

    @ManyToMany
    @JoinTable(
        name = "user_followers",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "follower_id")
    )
    private Set<User> followers = new HashSet<>();
}
```

Article 도메인

```java
@Entity
@Getter
@Setter
public class Article {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String slug;
    private String title;
    private String description;
    private String body;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    @ManyToOne
    private User author;

    @ManyToMany
    @JoinTable(name = "article_favorites")
    private Set<User> favoritedBy = new HashSet<>();

    @ElementCollection
    private List<String> tagList = new ArrayList<>();
}
```

Article Service

```java
@Service
@RequiredArgsConstructor
public class ArticleService {
    private final ArticleRepository articleRepository;
    private final UserRepository userRepository;

    public ArticleResponse createArticle(String username, CreateArticleRequest request) {
        User author = userRepository.findByUsername(username)
            .orElseThrow(() -> new UserNotFoundException("User not found"));

        Article article = new Article();
        article.setTitle(request.getTitle());
        article.setDescription(request.getDescription());
        article.setBody(request.getBody());
        article.setAuthor(author);
        article.setSlug(generateSlug(request.getTitle()));
        article.setTagList(request.getTagList());

        Article saved = articleRepository.save(article);
        return ArticleResponse.fromArticle(saved);
    }

    public Page<ArticleResponse> listArticles(Pageable pageable, String author, String tag) {
        Specification<Article> spec = (root, query, cb) -> cb.conjunction();

        if (author != null) {
            spec = spec.and((root, query, cb) ->
                cb.equal(root.get("author").get("username"), author)
            );
        }

        if (tag != null) {
            spec = spec.and((root, query, cb) ->
                cb.isMember(tag, root.get("tagList"))
            );
        }

        return articleRepository.findAll(spec, pageable)
            .map(ArticleResponse::fromArticle);
    }
}
```

실전 활용 시나리오

시나리오 1: 본인 기술스택 선택 후 확장

- Spring Boot 백엔드 + React 프론트엔드 조합으로 시작
- 댓글 기능, 실시간 알림, 고급 검색 추가
- 포트폴리오로 GitHub에 공개

시나리오 2: 데이터베이스 최적화 연습

- H2 → PostgreSQL 마이그레이션
- N+1 문제 해결 (Eager Loading vs Lazy Loading)
- 복합 인덱스 추가로 성능 개선

시나리오 3: 배포 파이프라인 구축
- Docker 컨테이너화
- GitHub Actions CI/CD
- 클라우드 배포 (AWS EC2, Heroku)

Claude Code 연동 팁

```
/think → RealWorld 아키텍처 분석
- Entity 관계 다이어그램 생성
- API 엔드포인트 설계 검토

/review → 코드 품질 검토
- JPA 쿼리 최적화 제안
- 에러 처리 개선 방안

/test → 테스트 케이스 자동 생성
- ArticleService 단위 테스트
- 통합 테스트 시나리오
```

3-1-2. Full Stack FastAPI Template

GitHub Stars: 28K+
프로젝트 설명: FastAPI + React + PostgreSQL + Docker 완전한 풀스택 보일러플레이트
용도: 빠른 프로젝트 시작, 현대적 스택 학습

개요

tiangolo/full-stack-fastapi-template는 프로덕션 레벨의 풀스택 템플릿입니다. Docker Compose로 모든 서비스를 한 번에 실행하고, JWT 인증, SQLAlchemy ORM, React 상태 관리를 포함합니다.

설치 방법

```
# 레포 클론
git clone https://github.com/tiangolo/full-stack-fastapi-template.git
cd full-stack-fastapi-template

# Docker Compose로 전체 스택 실행
docker-compose up -d

# 브라우저에서 확인
# 프론트엔드: http://localhost:3000
# API 문서: http://localhost:8000/docs
```

핵심 코드 구조

FastAPI User 모델

```python
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from pydantic import BaseModel

app = FastAPI()

class UserBase(BaseModel):
    email: str
    full_name: str = None

class UserCreate(UserBase):
    password: str

class User(UserBase):
    id: int
    is_active: bool

    class Config:
        from_attributes = True

@app.post("/api/v1/users/", response_model=User)
async def create_user(user: UserCreate, db: Session = Depends(get_db)):
    db_user = crud.create_user(db, user)
    return db_user
```

SQLAlchemy ORM

```python
from sqlalchemy import Column, Integer, String, Boolean, DateTime
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "user"

    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True, index=True)
    full_name = Column(String)
    hashed_password = Column(String)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

JWT 인증

```python
from fastapi.security import HTTPBearer, HTTPAuthCredentials
from jose import JWTError, jwt
from datetime import timedelta

security = HTTPBearer()

def create_access_token(data: dict, expires_delta: timedelta = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(hours=24)

    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(credentials: HTTPAuthCredentials = Depends(security)):
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=401)
    except JWTError:
        raise HTTPException(status_code=401)

    user = crud.get_user_by_email(db, username)
    return user
```

React + Axios 프론트엔드

```typescript
// src/api/users.ts
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:8000/api/v1';

interface User {
  id: number;
  email: string;
  full_name: string;
}

export const createUser = async (email: string, password: string): Promise<User> => {
  const response = await axios.post(`${API_URL}/users/`, {
    email,
    password,
    full_name: '',
  });
  return response.data;
};

export const getUser = async (token: string): Promise<User> => {
  const response = await axios.get(`${API_URL}/users/me`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });
  return response.data;
};
```

실전 활용 시나리오

시나리오 1: SaaS 애플리케이션 시작
- 기본 템플릿을 프로덕션 배포
- Stripe 결제 통합
- 사용자 조직 및 권한 관리 추가

시나리오 2: 마이크로서비스 분할
- FastAPI 백엔드를 여러 서비스로 분리
- 각 서비스별 독립적 배포
- API Gateway 구성

시나리오 3: 성능 최적화
- Redis 캐싱 계층 추가
- 데이터베이스 연결 풀 최적화
- CDN으로 정적 자산 배포

Claude Code 연동 팁

```
/plan-eng-review → 아키텍처 리뷰
- 마이크로서비스 분할 계획
- 확장성 제안

/qa → 테스트 자동 생성
- FastAPI 엔드포인트 테스트
- React 컴포넌트 테스트
```

3-1-3. Redoc: API 문서화 도구

GitHub Stars: 24K+
프로젝트 설명: OpenAPI 스펙을 아름다운 문서로 변환하는 도구
용도: API 문서 자동화, 개발자 경험 향상

개요

Redoc는 OpenAPI 3.0 스펙을 기반으로 인터랙티브한 API 문서를 자동 생성합니다. Swagger UI와 달리 Try It Out은 없지만, 매우 깔끔하고 모바일 친화적인 UI를 제공합니다.

설치 방법

```
# npm으로 설치
npm install redoc redoc-cli

# CLI로 정적 HTML 생성
npx redoc-cli build openapi.yaml -o index.html

# 또는 React 컴포넌트로 사용
npm install redoc react
```

핵심 코드 구조

OpenAPI 스펙 작성 (openapi.yaml)

RealWorld 스펙은 아래 요소를 중심으로 구성됩니다. 전체 YAML은 레포의 `api/` 폴더에서 바로 확인할 수 있습니다.

- `info`: title, version, description
- `servers`: Production/Staging URL
- `paths`: `/api/articles` (GET/POST), `/api/articles/{slug}`, `/api/users/login` 등
- `components.schemas`: Article, Profile, ArticleCreate (필수 필드: title/description/body)
- `components.securitySchemes`: BearerAuth (JWT)

React에서 Redoc 렌더링

```
import React from 'react';
import { ReDocStandalone } from 'redoc';

export const ApiDocumentation = () => {
  return (
    <ReDocStandalone
      specUrl="https://api.realworld.io/openapi.yaml"
      options={{
        scrollYOffset: 50,
        hideDownloadButton: false,
        hideHostname: false,
        hideLoading: false,
        suppressWarnings: false,
        expandResponses: '200,201',
      }}
    />
  );
};
```

FastAPI에서 자동 OpenAPI 생성

```
from fastapi import FastAPI
from fastapi.openapi.utils import get_openapi

app = FastAPI(
    title="RealWorld API",
    description="Medium Clone API",
    version="1.0.0",
)

def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema

    openapi_schema = get_openapi(
        title="RealWorld API",
        version="1.0.0",
        description="Medium Clone API",
        routes=app.routes,
    )

    openapi_schema["info"]["x-logo"] = {
        "url": "https://fastapi.tiangolo.com/img/logo-margin/logo-teal.png"
    }

    app.openapi_schema = openapi_schema
    return app.openapi_schema

app.openapi = custom_openapi

# http://localhost:8000/redoc 에서 문서 자동 렌더링
```

실전 활용 시나리오

시나리오 1: 자동 API 문서화

- FastAPI/Spring Boot 코드에서 자동 생성

- CI/CD 파이프라인에 통합
- 배포 시 최신 문서 자동 반영

시나리오 2: 개발자 포털 구축
- Redoc + 추가 가이드 페이지
- 인증 토큰 테스트
- 자동화된 SDK 생성

시나리오 3: 버전 관리
- 여러 API 버전 문서 병렬 운영
- 마이그레이션 가이드 제공

Claude Code 연동 팁

```
/think → OpenAPI 스펙 설계
- 엔드포인트 구조 분석
- 요청/응답 모델 최적화

/document-release → 자동 문서 생성
- OpenAPI YAML 자동 정리
- 배포용 HTML 생성
```

3-2. 포트폴리오 & 오픈소스 기여

3-2-1. First Contributions

GitHub Stars: 46K+
프로젝트 설명: 오픈소스 첫 기여 가이드 - PR 만드는 법 배우기
용도: 깃/GitHub 기초 학습, 오픈소스 기여 시작

개요

first-contributions/first-contributions는 오픈소스 첫 PR을 쉽게 만들 수 있도록 도와주는 튜토리얼입니다. 15개 이상 언어로 작성되었고, 간단한 PR 하나로 오픈소스 기여 경험을 얻을 수 있습니다.

설치 방법

```
# 1. 레포 Fork (GitHub 웹사이트에서)
# https://github.com/firstcontributions/first-contributions

# 2. 로컬에 클론
git clone https://github.com/<your-username>/first-contributions.git
cd first-contributions

# 3. upstream 리모트 추가
git remote add upstream https://github.com/firstcontributions/first-contributions.git
```

기본 워크플로우

Step 1: 브랜치 생성

```
git checkout -b add-<your-name>
```

Step 2: Contributors 파일에 이름 추가

`CONTRIBUTORS.md` 파일의 Contributors 섹션에 `- [Your Name](https://github.com/your-username)` 형식으로 한 줄 추가합니다.

Step 3: 변경 커밋

```
git add CONTRIBUTORS.md
git commit -m "Add <your-name> to Contributors"
```

Step 4: GitHub에 푸시 및 PR 생성

```
git push origin add-<your-name>
```

실전 활용 시나리오

시나리오 1: 첫 오픈소스 경험
- 이 레포에서 첫 PR 완성
- GitHub 프로필에 표시됨
- 오픈소스 문화 이해

시나리오 2: 오픈소스 기여 스킬 다졌기
- 더 큰 프로젝트로 진출
- 버그 리포트 → 픽스 → PR
- 포트폴리오 다양화

Claude Code 연동 팁

```
/think → Git 워크플로우 정리
- Fork vs Clone 차이
- Upstream sync 방법

/document-release → PR 템플릿 검토
- 설명 명확성 검증
- 커밋 메시지 개선
```

3-2-2. Awesome for Beginners

GitHub Stars: 70K+
프로젝트 설명: 초보자 친화 오픈소스 레포 모음
용도: 기여할 만한 레포 발견, 실전 기여 경험

개요

MunGell/awesome-for-beginners는 "good first issue" 라벨이 있는 오픈소스 레포들을 한 곳에 모아놨습니다. 각 레포마다 난이도, 언어, 설명이 있어 초보자가 쉽게 기여할 수 있는 프로젝트를 찾을 수 있습니다.

레포지토리 선택 가이드 (Python 초보자용 추천)

- **First Contributions** — 난이도 ★, 모든 언어, Git/GitHub 기본 학습
- **Up For Grabs** — 난이도 ★★, JavaScript/Python/C#, 초보자 태스크 수집 사이트
- **Exercism** — 난이도 ★★★, Python/JavaScript/Ruby 등, 코딩 연습 + 코드 리뷰
- **PySimpleGUI** — 난이도 ★★, Python, GUI 프로그래밍 입문

기여 전략

1단계: 좋은 이슈 찾기

```
# GitHub 검색
site:github.com label:"good first issue" language:python

# 또는 awesome-for-beginners에서 추천 레포 선택
```

2단계: 이슈 분석 체크리스트

- [ ] 이슈 설명을 완전히 이해했는가
- [ ] 요구 사항이 명확한가
- [ ] 내 스킬 레벨에 맞는가
- [ ] 관련 문서/가이드가 있는가

3단계: 코드 작성

```
# 예: 간단한 버그 픽스
# Before
def calculate_total(items):
    total = 0
    for item in items:
        total = total + item.price  # 버그: 할인 미적용
    return total

# After
def calculate_total(items):
    total = 0
    for item in items:
        total = total + (item.price * (1 - item.discount))
    return total
```

4단계: PR 작성

PR 본문에는 아래 항목이 포함되면 리뷰가 빨라집니다.

- 설명: 어떤 이슈(#123)를 해결하는지
- 변경 사항: 핵심 수정 3-5줄 bullet
- 테스트 결과: `pytest -v` 등 실제 실행 결과
- 체크리스트: 테스트/문서/커밋 메시지 확인

실전 활용 시나리오

시나리오 1: 3개월 도전
- 매주 1개 레포 기여
- 다양한 프로젝트 경험
- 포트폴리오 빠른 구축

시나리오 2: 특정 기술 집중
- Python 데이터 분석 라이브러리 기여
- 관련 기술 깊이 있게 학습
- 해당 분야 전문가 이미지 구축

Claude Code 연동 팁

```
/think → 이슈 분석
- 요구사항 분해
- 구현 전략 수립

/review → PR 품질 검증
- 코드 스타일 검토
- 테스트 커버리지 확인

/qa → 자동 테스트 생성
- 엣지 케이스 테스트
- 통합 테스트 작성
```

3-2-3. GitHub Opensource Guide

GitHub Stars: 14K+
프로젝트 설명: 오픈소스 프로젝트 운영 가이드
용도: 오픈소스 프로젝트 시작, 커뮤니티 관리

개요

github/opensource.guide는 오픈소스 프로젝트를 시작하고 운영하는 방법을 다룹니다. 라이선스, 기여 가이드, 커뮤니티 구성, 지속성 확보 등을 포함합니다.

필수 문서 체크리스트

1. 기본 설정: README.md, LICENSE(MIT/Apache 2.0/GPL 등), .gitignore, CONTRIBUTING.md
2. 코드 구조: 명확한 폴더 구조, 설치 가이드, 사용 예제, API 문서
3. 커뮤니티: CODE_OF_CONDUCT.md, Issue 템플릿, PR 템플릿, Discussions
4. 자동화: GitHub Actions CI/CD, 자동 테스트, 문서 배포, 릴리스 자동화

핵심 문서 작성

README.md 템플릿 구성 요소

- 프로젝트 타이틀 + Shields.io 배지(Stars, License, Build 등)
- 한 줄 설명(tagline)
- 기능 목록(bullet)
- 설치 명령어 (bash 코드 블록)
- 사용 예제 (주 언어 코드 블록)
- 기여 안내 (CONTRIBUTING.md 링크)
- 라이선스 표기

CONTRIBUTING.md 구성 요소

- 개발 환경 설정(clone/install/dev 명령)
- 커밋 메시지 규칙: Conventional Commits (`feat:`, `fix:`, `docs:`, `style:`, `refactor:`, `test:`, `chore:`)
- PR 프로세스: Fork → 브랜치 생성 → 커밋 → PR → 리뷰 → 병합

CODE_OF_CONDUCT.md 구성 요소

- 우리의 약속(존중, 건설적 피드백)
- 허용되지 않는 행동(괴롭힘, 차별, 욕설)
- 위반 보고 연락처

> Claude Code 활용 팁
> `> "github/opensource.guide를 참고해서 내 프로젝트(주제: xxx)용 README/CONTRIBUTING/CODE_OF_CONDUCT 초안을 만들어줘"`

실전 활용 시나리오

시나리오 1: 개인 프로젝트 오픈소스화
- 실무 프로젝트를 GitHub에 공개
- 가이드 따라 필수 문서 작성
- 첫 기여자 모으기

시나리오 2: 팀 프로젝트 관리
- 자동화된 이슈/PR 관리
- 릴리스 프로세스 정립
- 커뮤니티 성장 전략

Claude Code 연동 팁

```
/document-release → 문서 자동화
- README 템플릿 생성
- API 문서 자동 추출
- 릴리스 노트 작성

/cso → 보안 및 라이선스 검토
- 라이선스 호환성 확인
- 보안 정책 수립
```

3-3. 해커톤 & 공모전

3-3-1. Hackathon Starter

GitHub Stars: 35K+
프로젝트 설명: Node.js + Express + MongoDB 해커톤 보일러플레이트
용도: 해커톤 빠른 시작, Passport.js 인증 학습

개요

sahat/hackathon-starter는 웹 앱 기초(회원가입, 로그인, 소셜 로그인, API)를 포함한 완성된 스타터 템플릿입니다. 해커톤에서 2-3시간 내에 기본 백엔드를 완성하고 핵심 기능 개발에 집중할 수 있습니다.

설치 및 설정

```
# 클론
git clone https://github.com/sahat/hackathon-starter.git
cd hackathon-starter

# 의존성 설치
npm install

# .env 파일 설정
cp .env.example .env

# MongoDB 연결 (로컬 또는 MongoDB Atlas)
# MONGODB_URI=mongodb://localhost/hackathon

# 개발 서버 실행
npm run dev
```

핵심 코드: 인증

Passport 설정
- `config/passport.js`에서 `LocalStrategy`로 email/password 인증을 설정하고, `serializeUser`/`deserializeUser`로 세션을 유지합니다.
- 레포의 해당 파일을 그대로 복사해서 쓸 수 있도록 정리되어 있어 해커톤에서 시간 낭비가 없습니다.

User 모델
- `models/User.js`에서 Mongoose 스키마로 email/password/profile/provider를 정의합니다.
- `pre('save')` 훅으로 bcrypt 해싱, `comparePassword` 메서드로 로그인 검증.

라우트 및 컨트롤러
- `/signup`, `/login`, `/logout`, `/profile` (GET/PUT) 엔드포인트가 `routes/user.js`에 모두 구현돼

있습니다.
- 인증 보호는 `ensureAuthenticated` 미들웨어 하나로 처리.

소셜 로그인 (Google)
- `passport-google-oauth20`를 추가하고 `GOOGLE_ID`/`GOOGLE_SECRET` 환경변수만 설정하면 됩니다.
- 콜백 URL은 `/auth/google/callback`으로 고정, 레포에 Facebook/GitHub/Twitter 전략도 함께 들어 있습니다.

> Claude Code 활용 팁
> `> "hackathon-starter의 passport 설정을 내 프로젝트에 맞게 이메일 인증만 쓰는 버전으로 축약해줘"`

실전 활용 시나리오

시나리오 1: 24시간 해커톤
- 템플릿으로 1시간 내 기본 앱 완성
- 나머지 23시간 핵심 기능(AI, 데이터 분석 등) 개발
- 우승 가능성 높음

시나리오 2: API 해커톤
- 기본 인증 구조 재사용
- 다양한 API 엔드포인트 추가
- 빠른 프로토타입 배포

시나리오 3: 학습용
- Node.js + Express 기초 학습
- Passport.js 인증 패턴 이해
- MongoDB 스키마 설계 학습

Claude Code 연동 팁

```
/think → 해커톤 빠른 시작
- 필수 기능 분석
- 스케줄링

/plan-eng-review → 아키텍처 최적화
- API 엔드포인트 설계
- 데이터 모델 검증

/qa → 엣지 케이스 테스트
- 인증 플로우 테스트
- 에러 처리 검증
```

3-3-2. AutoGPT

GitHub Stars: 173K+
프로젝트 설명: 자율 AI 에이전트 구현 - OpenAI API 활용
용도: AI 해커톤, LLM 프로젝트, 자동화 학습

개요

Significant-Gravitas/AutoGPT는 "AI 자신이 목표 달성을 위해 자동으로 작업을 수행"하는 개념의 구현입니다. 사용자가 목표를 주면 AI가 필요한 단계들을 스스로 계획하고 실행합니다. 최근 버전은 프로덕션 안정성을 강화했습니다.

설치

```
# 클론
git clone https://github.com/Significant-Gravitas/AutoGPT.git
cd AutoGPT

# Python 환경
python -m venv venv
source venv/bin/activate  # 또는 venv\Scripts\activate (Windows)

# 의존성 설치
pip install -r requirements.txt

# OpenAI API 키 설정
export OPENAI_API_KEY="sk-..."

# 실행
python -m autogpt
```

핵심 아키텍처

Agent 패턴
- LangChain의 `initialize_agent`로 LLM + Tools + Memory를 결합한 에이전트를 만듭니다.
- Tool은 `(name, func, description)` 삼총사로 등록합니다. 예: Calculator, Search, FileWrite.
- `agent.run(goal)` 한 줄로 목표를 던지면 에이전트가 스스로 도구를 선택해 실행합니다.

Task Planning
- `PromptTemplate` + `LLMChain`으로 "목표 → 단계별 계획" 변환을 구현합니다.
- 결과 텍스트를 줄 단위 파싱해 숫자로 시작하는 줄만 steps 리스트에 담으면 간단합니다.

Memory System
- `ConversationBufferMemory`는 최근 대화를 그대로 기억, `ConversationSummaryMemory`는 요약해서 기억합니다.
- AutoGPT는 두 메모리를 함께 써서 긴 세션에서도 맥락을 유지합니다.

> Claude Code 활용 팁
> `> "AutoGPT의 Agent 클래스를 읽고 내 프로젝트(주제 검색 → 마크다운 리포트 생성)에 맞게 최소 버전으로 줄여줘"`

실전 활용 시나리오

시나리오 1: AI 해커톤 우승
- AutoGPT 기반 자동화 도구 개발
- 특정 도메인(예: 데이터 분석, 콘텐츠 생성) 최적화
- 사용자 피드백 반영 개선

시나리오 2: 작업 자동화
- 일일 리포트 자동 생성
- 이메일 응답 자동화
- 데이터 처리 파이프라인 구축

시나리오 3: 교육/포트폴리오
- LLM 기반 프로젝트 경험 제시
- AI 공학 기술 증명
- AI 회사 면접 준비

Claude Code 연동 팁

```
/think → 에이전트 설계
- Task planning 흐름
- Memory system 최적화
- Tool integration 전략

/review → 프롬프트 엔지니어링
- Few-shot 예제 추가
- 출력 형식 명확화
- Hallucination 방지

/qa → 테스트 자동화
- 에이전트 응답 검증
- 오류 처리 테스트
- 한국어 처리 검증
```

3-3-3. LangChain

GitHub Stars: 100K+
프로젝트 설명: LLM 애플리케이션 프레임워크
용도: AI 해커톤, RAG 시스템, LLM 체인 구축

개요

langchain-ai/langchain은 LLM(Large Language Model) 기반 애플리케이션을 빠르게 개발하는 프레임워크입니다. Prompt 관리, 메모리, 검색, 도구 통합을 통합적으로 제공합니다.

설치

```
pip install langchain openai python-dotenv
```

기본 사용법

Simple Chain

```python
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

llm = OpenAI(temperature=0)

# 템플릿 정의
prompt = PromptTemplate(
    input_variables=["topic"],
    template="다음 주제에 대해 설명하시오: {topic}"
)

# 체인 생성
chain = LLMChain(llm=llm, prompt=prompt)

# 실행
result = chain.run("머신러닝")
print(result)
```

Document Processing with RAG

```python
from langchain.document_loaders import PDFLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA

# PDF 로드
loader = PDFLoader("document.pdf")
documents = loader.load()

# 텍스트 분할
text_splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
docs = text_splitter.split_documents(documents)

# 벡터 임베딩
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(docs, embeddings)

# RAG 체인 생성
qa = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    chain_type="stuff",
    retriever=db.as_retriever()
)

# 문서 기반 질의응답
answer = qa.run("문서의 주요 내용은?")
print(answer)
```

Agent with Tools

```python
from langchain.agents import initialize_agent, Tool
from langchain.tools import DuckDuckGoSearchRun
from langchain.llms import OpenAI

# 도구 정의
search = DuckDuckGoSearchRun()

tools = [
    Tool(
        name="Search",
        func=search.run,
        description="웹 검색 수행"
    ),
    Tool(
        name="Calculator",
        func=lambda x: eval(x),
        description="수학 계산"
    )
]

# 에이전트 초기화
agent = initialize_agent(
    tools,
    OpenAI(temperature=0),
    agent="zero-shot-react-description",
    verbose=True
)

# 실행
result = agent.run("2024년 한국 GDP 성장률은?")
```

Memory Management

```python
from langchain.memory import ConversationBufferMemory, ConversationSummaryMemory
from langchain.chains import ConversationChain

# 메모리 설정
memory = ConversationBufferMemory()

# 대화형 체인
conversation = ConversationChain(
    llm=OpenAI(),
    memory=memory,
    verbose=True
)

# 연속 대화
conversation.predict(input="안녕, 나는 데이터 과학자야")
conversation.predict(input="나의 경력을 요약해줄래?")
```

Custom Chain

```python
from langchain.schema import BaseMemory
from langchain.chains import Chain

class CustomChain(Chain):
    @property
    def input_keys(self):
        return ["user_input"]

    @property
    def output_keys(self):
        return ["response"]

    def _call(self, inputs):
        user_input = inputs["user_input"]

        # 커스텀 로직
        response = f"사용자가 입력한 내용: {user_input}"

        return {"response": response}

# 사용
chain = CustomChain()
result = chain({"user_input": "안녕하세요"})
```

실전 활용 시나리오

시나리오 1: 챗봇 개발
- 컨텍스트 유지 메모리
- 다양한 도구 통합
- 자연스러운 대화 흐름

시나리오 2: 문서 분석 시스템
- RAG로 대규모 문서 처리
- 정확한 인용 기능
- 다중 언어 지원

시나리오 3: 해커톤 프로젝트
- 빠른 프로토타입 개발
- API 통합 간소화
- 복잡한 워크플로우 체인화

Claude Code 연동 팁

```
/think → 아키텍처 설계
- Chain 구성 전략
- Tool selection
- Memory strategy

/review → 프롬프트 최적화
- Temperature 조절
- Few-shot 예제 추가
- 출력 형식 지정

/qa → 통합 테스트
- 체인 정확성 검증
- 도구 호출 검증
```

이어서 카테고리 3를 작성하겠습니다.

Part 4: 문서 처리 라이브러리 완전 가이드

> 이 파트는 가이드북의 핵심입니다. 여기서 소개하는 라이브러리들은 이 가이드북 자체를 생성하는 데 실제로 사용되었습니다.
> `scripts/` 디렉토리의 빌드 파이프라인이 바로 이 라이브러리들의 실전 활용 예제입니다.

학습 목표

- Python/Java/JavaScript 문서 처리 라이브러리 활용
- Word, Excel, PDF, PowerPoint 자동 생성
- 문서 자동화 파이프라인 구축

4-1. Python 문서 라이브러리

4-1-1. python-docx (Word 문서)

설치

```
pip install python-docx
```

핵심 API

```python
from docx import Document
from docx.shared import Inches
from docx.enum.text import WD_ALIGN_PARAGRAPH

doc = Document()
title = doc.add_heading('프로젝트 리포트', 0)
title.alignment = WD_ALIGN_PARAGRAPH.CENTER
doc.add_paragraph('작성일: 2024-04-06')
doc.add_heading('1. 개요', level=1)
doc.add_paragraph('본문...', style='List Number')

table = doc.add_table(rows=2, cols=2)
table.style = 'Light Grid Accent 1'
table.rows[0].cells[0].text = '날짜'
table.rows[0].cells[1].text = '진행상황'

doc.add_picture('chart.png', width=Inches(5))
doc.save('report.docx')
```

외워둘 것: `add_heading(text, level)`, `add_paragraph(text, style)`, `add_table(rows, cols)`, `add_picture(path, width)`, `save(path)`. 동적 보고서는 dict/list를 순회하며 위 호출을 반복하면 끝입니다.

4-1-2. openpyxl (Excel 처리)

설치

```bash
pip install openpyxl
```

핵심 API

```python
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment

wb = Workbook()
ws = wb.active
ws.title = "Sales"

ws.append(['날짜', '상품', '판매량', '가격', '합계'])
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
    cell.fill = PatternFill("solid", fgColor="4472C4")
    cell.alignment = Alignment(horizontal="center")

ws.append(['2024-01-01', 'A상품', 10, 50000, None])
ws['E2'] = '=C2*D2'                # 수식
ws.column_dimensions['A'].width = 12
wb.save('sales.xlsx')
```

외워둘 것: `ws.append(row)`, `ws[f'E{r}'] = value`, `ws.column_dimensions[...].width`, `Font/PatternFill/Alignment`.

Excel 읽기 및 분석

```
from openpyxl import load_workbook
import statistics

wb = load_workbook('sales.xlsx')
ws = wb.active

# 데이터 읽기
data = []
for row in ws.iter_rows(min_row=2, values_only=True):
    if row[0]:  # 헤더 제외
        data.append(row)

# 분석
sales = [row[4] for row in data]  # 합계 열
print(f"총 판매액: {sum(sales)}")
print(f"평균 판매액: {statistics.mean(sales):.0f}")
print(f"중간값: {statistics.median(sales):.0f}")

# 필터링
high_sales = [row for row in data if row[4] > 500000]
print(f"500,000 이상 거래: {len(high_sales)}건")
```

피벗 테이블

```
from openpyxl import load_workbook
from openpyxl.worksheet.pivot import Pivot

wb = load_workbook('sales.xlsx')
ws = wb.active

# 데이터 범위
pivot = Pivot()
pivot.fromSheet = ws
pivot.ref = f'A1:E{ws.max_row}'

# 피벗 테이블 생성
pivot_sheet = wb.create_sheet('Pivot')
pivot.location = f'{pivot_sheet.title}!A1'

wb.save('sales_pivot.xlsx')
```

4-1-3. python-pptx (PowerPoint)

설치

```
pip install python-pptx
```

핵심 API

```python
from pptx import Presentation
from pptx.util import Inches
from pptx.chart.data import CategoryChartData
from pptx.enum.chart import XL_CHART_TYPE

prs = Presentation()

# 제목 슬라이드 (layout 0)
slide = prs.slides.add_slide(prs.slide_layouts[0])
slide.shapes.title.text = "프로젝트 발표"
slide.placeholders[1].text = "2024 Q1"

# 차트 슬라이드 (layout 5 = Blank)
slide = prs.slides.add_slide(prs.slide_layouts[5])
chart_data = CategoryChartData()
chart_data.categories = ['1월', '2월', '3월']
chart_data.add_series('판매액', (100, 120, 150))
slide.shapes.add_chart(XL_CHART_TYPE.COLUMN_CLUSTERED,
                       Inches(1), Inches(1), Inches(8), Inches(5), chart_data)

prs.save('presentation.pptx')
```

외워둘 것: `slide_layouts[0]`(Title), `[1]`(Title+Content), `[5]`(Blank). `add_textbox`, `add_picture`, `add_chart`로 레이아웃을 직접 쌓을 수 있습니다.

4-1-4. PyMuPDF (PDF 처리)

설치

```bash
pip install PyMuPDF
```

PDF 읽기 및 추출

```python
import fitz  # PyMuPDF

# PDF 열기
pdf_document = fitz.open('document.pdf')

# 기본 정보
print(f"페이지 수: {len(pdf_document)}")

# 텍스트 추출
for page_num in range(len(pdf_document)):
    page = pdf_document[page_num]
    text = page.get_text()
    print(f"--- 페이지 {page_num + 1} ---")
    print(text)

# 이미지 추출
for page_num in range(len(pdf_document)):
    page = pdf_document[page_num]
    images = page.get_images()

    for img_index, img in enumerate(images):
        xref = img[0]
        pix = fitz.Pixmap(pdf_document, xref)

        if pix.n - pix.alpha < 4:
            pix.save(f"page{page_num}_img{img_index}.png")
        else:
            pix = fitz.Pixmap(fitz.csRGB, pix)
            pix.save(f"page{page_num}_img{img_index}.png")

pdf_document.close()
```

PDF 생성

```python
import fitz

# 새 PDF 생성
doc = fitz.open()

# 페이지 추가
page = doc.new_page()

# 텍스트 추가
page.insert_text((50, 50), "안녕하세요", fontsize=12)

# 사각형 그리기
page.draw_rect((50, 80, 200, 120), color=(0, 0, 1), width=2)

# 이미지 삽입
page.insert_image((50, 150, 300, 350), filename="chart.png")

# 저장
doc.save('generated.pdf')
```

4-2. Java 문서 라이브러리

4-2-1. Apache POI (Excel/Word/PowerPoint)

Maven 의존성

```
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi</artifactId>
    <version>5.2.3</version>
</dependency>
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.3</version>
</dependency>
```

Excel 핵심 API (XSSFWorkbook)

```
Workbook wb = new XSSFWorkbook();
Sheet sheet = wb.createSheet("Sales");

Row header = sheet.createRow(0);
String[] cols = {"날짜", "상품", "판매량", "가격"};
for (int i = 0; i < cols.length; i++) {
    header.createCell(i).setCellValue(cols[i]);
}

Row row = sheet.createRow(1);
row.createCell(0).setCellValue("2024-01-01");
row.createCell(2).setCellValue(10);

try (FileOutputStream fos = new FileOutputStream("sales.xlsx")) {
    wb.write(fos);
}
wb.close();
```

읽기는 `WorkbookFactory.create(new File(...))` → `sheet.getSheetAt(0)` → `for (Row r : sheet)` 순서. 스타일은 `CellStyle` + `Font` + `FillPatternType.SOLID_FOREGROUND`.

Word 핵심 API (XWPFDocument)

```java
XWPFDocument doc = new XWPFDocument();

XWPFParagraph title = doc.createParagraph();
title.setAlignment(ParagraphAlignment.CENTER);
XWPFRun run = title.createRun();
run.setText("프로젝트 리포트");
run.setBold(true);
run.setFontSize(18);

XWPFTable table = doc.createTable(3, 2);
table.getRow(0).getCell(0).setText("날짜");
table.getRow(0).getCell(1).setText("진행상황");

try (FileOutputStream fos = new FileOutputStream("report.docx")) {
    doc.write(fos);
}
doc.close();
```

외워둘 것: 모든 텍스트는 `Paragraph → Run` 2단계로 추가. 스타일은 Run에 걸립니다.

4-2-2. iText (PDF 생성/편집)

Maven 의존성

```pom.xml
<dependency>
    <groupId>com.itextpdf</groupId>
    <artifactId>itext7-core</artifactId>
    <version>7.2.3</version>
    <type>pom</type>
</dependency>
```

PDF 생성

```java
import com.itextpdf.kernel.pdf.PdfDocument;
import com.itextpdf.kernel.pdf.PdfWriter;
import com.itextpdf.layout.Document;
import com.itextpdf.layout.element.Paragraph;
import com.itextpdf.layout.element.Table;
import java.io.IOException;

public class PdfGenerator {
    public static void createPdf() throws IOException {
        String dest = "output.pdf";

        PdfWriter writer = new PdfWriter(dest);
        PdfDocument pdfDoc = new PdfDocument(writer);
        Document document = new Document(pdfDoc);

        // 제목
        document.add(new Paragraph("프로젝트 리포트")
            .setBold()
            .setFontSize(18));

        // 본문
        document.add(new Paragraph("작성일: 2024년 4월 6일"));

        // 테이블
        Table table = new Table(2);
        table.addCell("날짜");
        table.addCell("진행상황");
        table.addCell("2024-01-01");
        table.addCell("기획");
        table.addCell("2024-02-01");
        table.addCell("개발");

        document.add(table);
        document.close();
    }
}
```

4-3. JavaScript 문서 라이브러리

4-3-1. SheetJS (xlsx)

설치

```
npm install xlsx
```

기본 사용법

```
import XLSX from 'xlsx';

// Excel 파일 읽기
const file = input.files[0];
const reader = new FileReader();

reader.onload = (e) => {
    const data = new Uint8Array(e.target.result);
    const workbook = XLSX.read(data, { type: 'array' });
    const worksheet = workbook.Sheets[workbook.SheetNames[0]];
    const json = XLSX.utils.sheet_to_json(worksheet);

    console.log(json);
};

reader.readAsArrayBuffer(file);

// Excel 파일 생성
const data = [
    { 날짜: '2024-01-01', 상품: 'A상품', 판매량: 10 },
    { 날짜: '2024-01-02', 상품: 'B상품', 판매량: 15 },
];

const worksheet = XLSX.utils.json_to_sheet(data);
const workbook = XLSX.utils.book_new();
XLSX.utils.book_append_sheet(workbook, worksheet, "Sales");
XLSX.writeFile(workbook, "sales.xlsx");
```

4-3-2. PDFKit

설치

```
npm install pdfkit
```

PDF 핵심 API

```
const PDFDocument = require('pdfkit');
const fs = require('fs');

const doc = new PDFDocument();
doc.pipe(fs.createWriteStream('output.pdf'));

doc.fontSize(25).text('프로젝트 리포트', { align: 'center' });
doc.fontSize(12).text('작성일: 2024-04-06');
doc.image('chart.png', { width: 400 });
doc.addPage().text('두 번째 페이지');

doc.end();
```

외워둘 것: `doc.text()`는 연쇄 호출 가능. 테이블은 좌표 계산(`x + i * width`)으로 직접 그립니다.

4-3-3. docx (Word 생성)

설치

```
npm install docx
```

Word 핵심 API

```
import { Document, Packer, Paragraph, Table, TableRow, TableCell } from "docx";
import fs from "fs";

const cell = (text) => new TableCell({ children: [new Paragraph(text)] });

const doc = new Document({
  sections: [{
    children: [
      new Paragraph({ text: "프로젝트 리포트", bold: true, size: 28 }),
      new Paragraph({ text: "작성일: 2024-04-06" }),
      new Table({
        rows: [
          new TableRow({ children: [cell("날짜"), cell("진행상황")] }),
          new TableRow({ children: [cell("2024-01-01"), cell("기획")] }),
        ],
      }),
    ],
  }],
});

Packer.toBuffer(doc).then((buf) => fs.writeFileSync("report.docx", buf));
```

외워둘 것: `Document` → `sections[].children[]`에 `Paragraph`/`Table`을 선언형으로 쌓는 패턴. `Packer.toBuffer()`로 바이너리를 얻어 저장합니다.

4-4. 문서 자동화 파이프라인

Claude Code와 문서 라이브러리 통합

종합 문서 생성 워크플로우

이 가이드북의 `scripts/build_guidebook.py`가 실제 예시입니다. 구조는 단순합니다.

```python
class DocumentPipeline:
    def __init__(self, data):
        self.data = data

    def generate_all(self):
        self._word()      # python-docx
        self._excel()     # openpyxl
        self._pptx()      # python-pptx
```

각 `_word`, `_excel`, `_pptx` 메서드는 앞서 본 핵심 API를 그대로 호출합니다. 입력을 dict 하나로 통일하면 같은 데이터로 세 포맷이 동시에 나옵니다.

> Claude Code 활용 팁
> `> "내 프로젝트 데이터(dict)를 받아서 docx/xlsx/pptx 3종 문서를 한 번에 생성하는 DocumentPipeline 클래스를 만들어줘"`

실전 활용: 포트폴리오 자동 생성

GitHub API + python-docx를 결합해 "내 Top 10 레포" 포트폴리오를 자동으로 만드는 스크립트를 작성할 수 있습니다.

```python
import requests
from docx import Document

def generate_portfolio(username):
    repos = requests.get(f'https://api.github.com/users/{username}/repos').json()
    repos = sorted(repos, key=lambda r: r['stargazers_count'], reverse=True)[:10]

    doc = Document()
    doc.add_heading(f'{username} 포트폴리오', 0)
    for repo in repos:
        doc.add_heading(repo['name'], level=1)
        doc.add_paragraph(repo.get('description') or '')
        doc.add_paragraph(f"⭐ {repo['stargazers_count']} | {repo.get('language') or 'N/A'}")
        doc.add_paragraph(repo['html_url'])
    doc.save(f'{username}_portfolio.docx')

generate_portfolio('torvalds')
```

> Claude Code 활용 팁
> `> "위 스크립트를 내 GitHub 아이디로 실행해서 Top 10 레포 포트폴리오를 Word로 뽑아줘. 각 레포마다 README 첫 문단도 요약해서 넣어줘."`

Part 5: AI 개발 도구 — Claude Code MCP & GSTACK

> Claude Code의 MCP 서버 생태계와 GSTACK 워크플로우를 통합하여 다룹니다.
> AI 도구를 활용한 개발 생산성 극대화 전략을 배울 수 있습니다.

학습 목표

- MCP 서버 개념과 핵심 서버 활용
- GSTACK 23개 스킬 실전 활용
- Plan 모드 & Sequential Thinking 활용법
- AI 도구 조합 전략

Claude Code와 통합되는 Model Context Protocol(MCP) 서버들을 체계적으로 설정하고 활용하는 방법입니다.

5-1. 핵심 MCP 서버

MCP 소개

Model Context Protocol(MCP)은 Claude Code에서 외부 도구와 데이터에 접근하기 위한 표준 프로토콜입니다.

```
구조:
Claude Code
    ↓
MCP Client
    ↓
MCP Server (GitHub, Filesystem, PostgreSQL 등)
    ↓
External Services
```

GitHub MCP - 레포지토리 관리

설치

```
claude mcp add github -s user -- npx -y @modelcontextprotocol/server-github
```

설정 (`~/.claude/config.json`)

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "ghp_your_token_here"
      }
    }
  }
}
```

활용 사례

```
GitHub MCP로 가능한 작업:
1. 레포지토리 검색 및 정보 조회
2. Issue 생성, 조회, 수정
3. Pull Request 관리
4. 파일 조회 및 수정 제안
5. 커밋 이력 확인

사용 예시:
- "kubernetes/kubernetes 레포의 최신 이슈 10개 보여줘"
- "tensorflow/tensorflow의 PR #12345 내용 확인"
- "pytorch/pytorch에서 'performance' 라벨이 있는 이슈들 찾아줘"
```

Filesystem MCP - 로컬 파일 시스템

설치

```
claude mcp add filesystem -s local -- npx -y @modelcontextprotocol/server-filesystem /path/to/workspace
```

설정

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/home/user/projects"
      ]
    }
  }
}
```

활용 시나리오

```
Filesystem MCP로 가능한 작업:
1. 프로젝트 파일 구조 탐색
2. 코드 파일 읽기/쓰기
3. 설정 파일 관리
4. 로그 파일 분석
5. 대량 파일 처리

사용 예시:
- "src 디렉토리의 모든 Python 파일 목록 보여줘"
- "config.json 파일을 읽고 수정해줘"
- ".log 파일들을 분석해서 에러 패턴 찾아줘"
```

PostgreSQL MCP - 데이터베이스 연동

설치

```
claude mcp add postgres -s user -- npx -y @modelcontextprotocol/server-postgres
```

설정

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://user:password@localhost:5432/mydb"
      }
    }
  }
}
```

활용 예제

```sql
-- MCP를 통해 Claude Code에서 직접 실행
-- 1. 테이블 스키마 확인
SELECT * FROM information_schema.tables
WHERE table_schema = 'public';

-- 2. 데이터 조회
SELECT * FROM users
WHERE created_at > NOW() - INTERVAL '7 days'
ORDER BY created_at DESC;

-- 3. 분석 쿼리
SELECT
    DATE(created_at) as date,
    COUNT(*) as user_count,
    AVG(age) as avg_age
FROM users
GROUP BY DATE(created_at)
ORDER BY date DESC;

-- Claude Code의 분석:
-- "지난 주 일일 사용자 수와 평균 나이를 조회했습니다.
--  데이터에 따르면 월요일에 가장 많은 신규 사용자가 가입했습니다."
```

Sequential Thinking MCP - 복잡한 분석

설치

```
claude mcp add sequential-thinking -s local -- npx -y @modelcontextprotocol/server-sequential-thinking
```

설정

```json
{
  "mcpServers": {
    "sequential-thinking": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-sequential-thinking"]
    }
  }
}
```

활용 사례

```
복잡한 분석이 필요한 경우:

1. 시스템 아키텍처 설계
   "우리 마이크로서비스 시스템을 PostgreSQL에서 MongoDB로 마이그레이션하려고 합니다.
    단계별로 어떻게 해야 할까요?"

   → Sequential Thinking으로 각 단계 논리적 분석

2. 코드 리팩토링 계획
   "이 12,000줄의 모놀리식 애플리케이션을 마이크로서비스로 분해하는
    최선의 방법을 알려주세요."

   → 각 컴포넌트별 분석 및 마이그레이션 전략 수립

3. 성능 최적화
   "데이터베이스 쿼리가 느립니다. 어디서 최적화해야 할까요?"

   → 쿼리 분석 → 인덱싱 전략 → 캐싱 전략 → 구현 단계 제시
```

5-2. 생산성 MCP 서버

Playwright MCP - 브라우저 자동화

설치

```
claude mcp add playwright -s user -- npx -y @modelcontextprotocol/server-playwright
```

활용 시나리오

```
웹 자동화 작업:

1. 웹 스크래핑
   "airbnb.com에서 서울의 모든 숙박소 정보를 수집해줘"

2. 테스트 자동화
   "이 로그인 폼을 테스트해: username, password 입력 후 로그인"

3. 스크린샷 캡처
   "github.com의 스크린샷을 찍고 가독성을 평가해줘"

4. 폼 자동 작성
   "이 신청 폼을 내 정보로 채워줘"
```

Claude Code 대화 예시

> "GitHub에서 'machine learning' 키워드로 검색하고, 상위 10개 레포의 이름과 Stars 수를 표로 정리해줘."

Claude Code는 Playwright MCP를 통해 다음을 자동으로 수행합니다.
- 브라우저 실행 및 페이지 접속
- 검색어 입력 및 결과 탐색
- DOM 요소에서 데이터 추출
- 결과를 Markdown 표로 반환

> "이 로그인 폼이 잘못된 비밀번호일 때 어떤 에러 메시지를 보여주는지 확인해줘."

Claude Code가 시나리오를 실행한 뒤 스크린샷과 함께 결과를 요약합니다. 사용자는 코드를 직접 작성할 필요가 없습니다.

Memory MCP - 지속적인 지식 관리

설치

```
claude mcp add memory -s user -- npx -y @modelcontextprotocol/server-memory
```

활용

```
세션 간 정보 유지:

1. 프로젝트 컨텍스트 저장
   "이 프로젝트의 주요 결정사항들을 메모리에 저장해줘"

   저장 내용:
   - 프로젝트 구조
   - 핵심 알고리즘
   - 성능 최적화 전략
   - 향후 계획

2. 학습 진행도 추적
   "지금까지 배운 내용과 다음에 배워야 할 내용을 정리해줘"

3. 버그 레지스트리
   "발견한 버그들과 해결 방법을 저장해줘"
```

Docker MCP - 컨테이너 관리

설치

```
claude mcp add docker -s user -- npx -y @modelcontextprotocol/server-docker
```

활용 예제

```
# Docker 이미지 생성
docker build -t myapp:1.0 .

# 컨테이너 실행
docker run -d -p 8080:8080 --name myapp myapp:1.0

# 로그 확인
docker logs myapp

# 통계 보기
docker stats myapp

# Claude Code와의 연동:
# "이 Node.js 앱을 Docker화해줄래?"
# → Dockerfile 자동 생성
# → docker-compose.yml 생성
# → 이미지 빌드 및 실행
```

5-3. MCP 서버 조합 전략

개발 워크플로우 (완전한 예제)

```
목표: REST API 개발 프로젝트 자동화

1단계: 프로젝트 구조 생성 (Filesystem MCP)
├── src/
│   ├── main.py
│   ├── models.py
│   └── routes.py
├── tests/
│   └── test_api.py
├── docker/
│   └── Dockerfile
└── requirements.txt

2단계: GitHub에 레포 생성 (GitHub MCP)
├── 새 저장소 생성
├── README 작성
└── 초기 커밋

3단계: PostgreSQL 데이터베이스 설계 (PostgreSQL MCP)
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    email VARCHAR(100) UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

4단계: Docker 환경 설정 (Docker MCP)
docker-compose up -d  # PostgreSQL + API 서버

5단계: API 엔드포인트 구현
GET    /api/users      - 모든 사용자 조회
POST   /api/users      - 새 사용자 생성
GET    /api/users/:id  - 특정 사용자 조회
PUT    /api/users/:id  - 사용자 수정
DELETE /api/users/:id  - 사용자 삭제

6단계: 테스트 자동화
pytest tests/ -v
```

Claude Code 대화 예시 (MCP 조합 활용)

> "FastAPI + PostgreSQL로 사용자 관리 REST API 프로젝트를 만들어줘.
> - Filesystem MCP로 src/, tests/, docker/ 구조 생성
> - PostgreSQL MCP로 users 테이블 스키마 설계
> - GitHub MCP로 새 레포 생성 및 초기 커밋
> - Docker MCP로 docker-compose.yml 작성"

Claude Code가 각 MCP를 순차적으로 호출하여 프로젝트를 구성합니다. 사용자는 "생성된 models.py를 보여줘", "users 테이블에 role 컬럼을 추가해줘" 등 자연어로 후속 작업을 이어갈 수 있습니다.

핵심은 **사용자가 코드를 쓰지 않고 MCP에 작업을 위임**한다는 점입니다. Claude가 생성한 파일은 Plan 모드로 검토 후 승인할 수 있습니다.

문서 작업 워크플로우

```
목표: 기술 문서 자동 생성

1. Filesystem MCP: 프로젝트 코드 분석
2. Sequential Thinking MCP: 문서 구조 계획
3. GitHub MCP: 최신 변경사항 확인
4. Playwright MCP: 스크린샷 자동 캡처
5. 최종 문서 생성 (Markdown + PDF)
```

GSTACK은 Garry Tan의 23개 스킬 팩으로, Claude Code를 체계적으로 활용할 수 있게 합니다.

5-4. GSTACK 설치 및 설정

설치 방법

```
# GSTACK 설치
claude install-skill github.com/garrytan/gstack

# 설치 확인
claude --version  # Claude Code 버전 확인
claude skills list  # 설치된 스킬 목록
```

23개 GSTACK 스킬 설명

```
GSTACK Framework - 23 Skills

1️⃣ CEO / 전략 수립 (2개)
   /office-hours
   - 목적: 프로젝트 전략 상담
   - 사용 시기: 프로젝트 시작 또는 주요 결정 필요 시
   - 입력: 프로젝트 아이디어 또는 문제 상황
   - 출력: 전략적 조언, 우선순위 결정

   /plan-ceo-review
   - 목적: CEO 관점의 계획 리뷰
   - 사용 시기: 계획 수립 후 검토
   - 입력: 프로젝트 계획서
   - 출력: 비즈니스 임팩트 평가

2️⃣ Engineer / 기술 구현 (3개)
   /plan-eng-review
   - 목적: 엔지니어링 관점 기술 계획 리뷰
   - 확인 항목: 기술 스택 선택, 아키텍처 설계, 구현 가능성

   /review
   - 목적: 코드 품질 리뷰
   - 검토 항목: 코드 구조, 성능, 보안, 테스트 커버리지
   - 최적화 제안 제공

   /think
   - 목적: 깊은 기술적 사고와 분석
   - 사용 시기: 복잡한 기술 문제 해결 필요 시
   - Sequential Thinking MCP 활용

3️⃣ Designer / UX (2개)
   /design-consultation
   - 목적: UX/UI 전략 수립
   - 검토: 사용자 흐름, 접근성, 디자인 시스템

   /design-review
   - 목적: 디자인 상세 리뷰
   - 검토: 일관성, 가독성, 반응형 디자인

4️⃣ QA / 품질 보증 (2개)
   /qa
   - 목적: 종합 품질 보증 검사
   - 검토: 코드 + 테스트 + 성능
   - 결과: 테스트 코드 생성 및 개선안

   /qa-only
   - 목적: 테스트만 집중 실행
   - 결과: 테스트 커버리지 보고서

5️⃣ Release / 배포 (3개)
   /ship
   - 목적: 배포 준비 및 실행
   - 프로세스: 체크리스트 확인 → 배포 → 검증

   /land-and-deploy
   - 목적: 안전한 랜딩(병합) 및 배포
   - 프로세스: PR 리뷰 → 병합 → 배포 → 모니터링

   /canary
   - 목적: 카나리 배포 실행
```

```
   - 프로세스: 5% 사용자 → 25% → 100%

6️⃣ Security / 보안 (1개)
   /cso
   - 목적: 보안 최고 책임자(CSO) 관점 리뷰
   - 검토: 인증/인가, 데이터 보안, 규제 준수

7️⃣ Documentation (2개)
   /document-release
   - 목적: 릴리스 문서 자동 생성
   - 산출물: 변경사항, API 문서, 마이그레이션 가이드

   /retro
   - 목적: 회고 문서 작성
   - 산출물: 배운 점, 개선안, 다음 스프린트 계획

8️⃣ Planning & Analysis (7개)
   /weekly (주간 계획)
   /daily (일일 계획)
   /scope (범위 정의)
   /breakdown (작업 분해)
   /estimate (기간 산정)
   /risk (위험 분석)
   /deps (의존성 분석)
```

5-5. GSTACK 역할별 활용 가이드

CEO 역할: 전략 수립

시나리오: 새로운 SaaS 프로젝트 시작

```
# 1단계: /office-hours로 전략 수립
claude /office-hours
입력:
"AI 기반 문서 자동화 SaaS를 만들려고 합니다.
 타겟: 중소 기업 (1-50명)
 핵심 기능: PDF/Word 문서 자동 생성, 템플릿 관리
 예상 MRR: $5,000
 우리의 강점: Python 개발팀, 머신러닝 경험"

출력:
✓ 시장 분석 및 경쟁사 분석
✓ GTM(Go-to-Market) 전략
✓ 단계별 마일스톤 (MVP → v1 → Scale)
✓ 펀딩 전략
✓ KPI 및 성공 지표

# 2단계: /plan-ceo-review로 계획 검증
claude /plan-ceo-review < sprint_plan.md

입력: 스프린트 계획서
출력:
✓ 비즈니스 임팩트 평가
✓ 리소스 할당 검토
✓ 일정 현실성 검토
✓ 위험 요소 식별
✓ 우선순위 재조정
```

Engineer 역할: 기술 구현

시나리오: 복잡한 마이크로서비스 아키텍처 설계

```
# 1단계: /plan-eng-review로 기술 설계 검토
claude /plan-eng-review
입력:
"마이크로서비스 아키텍처 설계:
 - API Gateway (Kong)
 - User Service (Node.js)
 - Document Service (Python FastAPI)
 - AI Service (Python FastAPI + PyTorch)
 - Message Queue (RabbitMQ)
 - Database (PostgreSQL + Redis)"

출력:
✓ 기술 스택 타당성 검토
✓ 확장성/성능 분석
✓ 운영 복잡도 평가
✓ 마이그레이션 경로 제시
✓ 개선 제안

# 2단계: /think로 깊은 기술 분석
claude /think
입력: "AI Service의 레이턴시를 100ms 이하로 줄이려면?"

출력 (Sequential Thinking):
1⃣ 현재 병목 분석
   - 모델 로드 시간: 50ms
   - 추론 시간: 40ms
   - I/O 오버헤드: 10ms

2⃣ 해결 방안 개발
   - 모델 캐싱 (GPU 메모리)
   - 배치 처리
   - 요청 큐잉 최적화

3⃣ 구현 계획
   - 단계 1: 모델 캐싱 구현 (5일)
   - 단계 2: 배치 처리 추가 (3일)
   - 단계 3: 성능 테스트 및 모니터링 (2일)

4⃣ 예상 개선
   - 레이턴시: 100ms → 35ms 예상

# 3단계: /review로 코드 리뷰
claude /review < pull_request_123.diff

출력:
✓ 코드 품질 평가 (Pylint, SonarQube 기준)
✓ 성능 분석
✓ 보안 취약점 식별
✓ 테스트 커버리지 확인
✓ 개선 제안 + 수정 코드 제시
```

/review 활용 시나리오

> "/review 실행해줘. process_documents 함수가 for 루프 안에서 DB를 조회하고 있어서 느린 것 같아."

`/review` 스킬은 다음과 같은 관점에서 코드를 점검합니다.
- N+1 쿼리 패턴 감지 및 배치 쿼리 제안
- 동기 I/O를 비동기로 전환해야 할 부분 지적
- 에러 처리 및 로깅 누락 영역 표시
- 테스트 커버리지 부족 영역 리포트

리뷰 결과는 "Critical 2건, Warning 5건" 같은 요약과 함께 각 이슈에 대한 개선안을 제시합니다. 사용자는 "1번 이슈만 반영해줘"처럼 선택적으로 적용할 수 있습니다.

QA 역할: 품질 보증

```
# 1단계: /qa로 종합 품질 검사
claude /qa
입력: src/ 디렉토리

출력:
✓ 코드 품질 점수: 8.5/10
✓ 테스트 커버리지: 78%
✓ 발견된 이슈:
  - 5개 중위험 (수정 필요)
  - 12개 저위험 (개선 권장)
✓ 테스트 코드 자동 생성 (미싱 부분)

# 2단계: /qa-only로 테스트만 집중
claude /qa-only
입력: src/ 디렉토리

출력:
✓ 테스트 실행 결과: 127 passed, 3 failed
✓ 새 테스트 코드 생성 (커버리지 보충)
✓ 성능 테스트 리포트
✓ 권장 사항
```

Release 역할: 배포 관리

시나리오: v2.0 릴리스 배포

```
# 1단계: /ship로 배포 준비
claude /ship
입력:
"v2.0 배포:
 - Main branch commit: abc123...
 - Release notes: RELEASE_v2.0.md
 - Deployment target: Production"

출력:
배포 체크리스트:
☑ 모든 테스트 통과? ✓
☑ 코드 리뷰 완료? ✓
☑ 보안 검사 통과? ✓
☑ DB 마이그레이션 준비? ✓
☑ 롤백 계획 수립? ✓

배포 프로세스:
1⃣ Staging 배포 및 검증
2⃣ 성능 테스트 실행
3⃣ Production 배포
4⃣ 헬스 체크
5⃣ 모니터링 설정

예상 다운타임: 0분 (무중단 배포)

# 2단계: /canary로 카나리 배포
claude /canary
입력:
"단계별 배포:
 - Phase 1: 5% 트래픽 (1시간)
 - Phase 2: 25% 트래픽 (30분)
 - Phase 3: 100% 트래픽"

배포 모니터링:
Phase 1 (5%):
  - Error rate: 0.01% (정상)
  - Latency: +2% (양호)
  → Phase 2 진행

Phase 2 (25%):
  - Error rate: 0.02% (정상)
  - Latency: +3% (양호)
  - Memory usage: +5% (양호)
  → Phase 3 진행

Phase 3 (100%):
  ✓ 배포 완료
  ✓ 모든 메트릭 정상
```

5-6. GSTACK + MCP 시너지 워크플로우

완전한 프로젝트 라이프사이클

```
📋 프로젝트: "AI 기반 이메일 템플릿 생성기"
목표: 사용자가 자연어로 이메일 템플릿을 생성하고 관리하는 SaaS

====================================
1단계: 전략 수립 (1주)
====================================

CEO: /office-hours
├─ 시장 분석 (Gmail, Outlook 사용자 타겟)
├─ MVP 범위 정의 (기본 템플릿 생성 + 저장)
└─ 6개월 로드맵 수립

├─ /plan-ceo-review
  └─ 비즈니스 모델 검증 (구독형 vs 한번 구매)

====================================
2단계: 기술 설계 (2주)
====================================

Engineer: /plan-eng-review
├─ 기술 스택 선택
│  ├─ Frontend: Next.js (React + TypeScript)
│  ├─ Backend: Python FastAPI
│  ├─ AI: GPT-4 API
│  ├─ Database: PostgreSQL + Redis
│  └─ Infrastructure: Docker + Kubernetes
└─ 아키텍처 다이어그램 작성

├─ /think (Sequential Thinking)
  └─ LLM 프롬프트 최적화 전략
    1⃣ 현재 문제: 생성 속도 느림 (avg 5초)
    2⃣ 분석: 토큰 수 많음, 스트리밍 미사용
    3⃣ 솔루션: 스트리밍 + 프롬프트 최적화
    4⃣ 예상 개선: 5초 → 2초

Designer: /design-consultation
└─ UI/UX 프로토타입 검토

====================================
3단계: 개발 구현 (6주)
====================================

GitHub MCP + Filesystem MCP:
프로젝트 구조 자동 생성
├── frontend/
│   ├── components/
│   ├── pages/
│   └── styles/
├── backend/
│   ├── app/
│   │   ├── models.py
│   │   ├── routes.py
│   │   └── services.py
│   └── tests/
└── infra/
    ├── docker-compose.yml
    └── k8s/

Week 1-2: Backend API 개발
  Engineer: /review (매일)
```

```
  코드 품질 유지

Week 3-4: Frontend 개발
  Designer: /design-review
  UI 정확성 확인

Week 5-6: AI 통합
  Engineer: /think
  LLM API 최적화

====================================
4단계: 품질 보증 (2주)
====================================

QA: /qa
├ 자동 테스트 커버리지: 85%로 목표
├ 버그 식별 및 분류
└ 성능 테스트

QA: /qa-only
├ 엣지 케이스 테스트
├ 부하 테스트 (1000 concurrent users)
└ 보안 테스트

Security (CSO): /cso
├ API 인증/인가 검증
├ 데이터 암호화 확인
└ GDPR 규제 준수 검토

====================================
5단계: 배포 준비 (1주)
====================================

Release: /document-release
├ 변경사항 문서화
├ API 문서 자동 생성 (OpenAPI)
└ 마이그레이션 가이드

Release: /ship
├ 배포 체크리스트 확인
├ Staging 배포 및 검증
└ Production 배포

Release: /canary (Progressive Rollout)
├ Phase 1: 5% 사용자 (1시간)
├ Phase 2: 25% 사용자 (30분)
└ Phase 3: 100% 사용자

====================================
6단계: 운영 및 최적화 (진행 중)
====================================

모니터링 (PostgreSQL MCP + Memory MCP):
├ 일일 사용자 수
├ API 레이턴시
├ 에러율
└ AI 토큰 사용량

주간 /retro:
├ 배운 점 정리
```

```
├ 개선안 수집
└ 다음 스프린트 계획
```

5-7. Plan 모드 & Sequential Thinking 활용법

Plan 모드란?

Claude Code의 Plan 모드는 변경사항을 실제로 적용하기 전에 미리보기할 수 있는 기능입니다.

```
Plan Mode 장점:
✓ 예상치 못한 변경 방지
✓ 대량 파일 수정 시 검증
✓ 롤백 가능성 확보
✓ 팀 협업 시 코드 리뷰 용이
```

Plan 모드 사용법

```
# 1. Plan 모드 활성화
claude --plan

# 2. 변경사항 요청
"이 프로젝트를 TypeScript로 마이그레이션해줘"

# 3. Claude Code가 변경 계획 제시
Plan Preview:
├ 파일 생성: tsconfig.json
├ 파일 생성: src/**/*.ts (15개 파일)
├ 파일 수정: package.json
├ 파일 삭제: src/**/*.js (15개 파일)
└ 예상 소요 시간: 30분

# 4. 변경사항 확인 후 승인
"✓ 진행해줘"

# 5. 변경사항 적용
실제 파일 시스템에 변경 적용됨
```

Sequential Thinking 활용법

Sequential Thinking은 복잡한 문제를 단계별로 논리적으로 분석합니다.

시나리오: 성능 최적화 전략

```
claude /think
입력:
"우리 API의 평균 응답 시간이 500ms입니다.
 현재 스택: Node.js + Express + PostgreSQL
 일일 요청: 1000만 건
 목표: 응답 시간을 100ms 이하로 줄이기"

Sequential Thinking 프로세스:

====================================
Step 1: 문제 분석 (현재 상황 파악)
====================================

병목 지점 식별:
1. Express 라우팅: 10ms
2. 비즈니스 로직: 150ms
3. 데이터베이스 쿼리: 300ms
4. 직렬화/응답: 40ms

가장 큰 문제: 데이터베이스 쿼리 (300ms = 60%)

====================================
Step 2: 원인 분석 (왜?)
====================================

DatabaseQuery 느린 이유:
1. 인덱스 부족
   - 자주 쿼리되는 컬럼에 인덱스 없음
   - 스캔 시간: 200ms

2. 캐싱 부재
   - 동일 쿼리 반복 실행
   - 불필요한 DB 부하

3. N+1 쿼리 문제
   - 부모 1개 쿼리 + 자식 100개 쿼리
   - 누적 시간: 100ms

====================================
Step 3: 해결 방안 수립
====================================

단기 (1주):
1. 핫 테이블에 인덱스 추가
   - 예상 개선: 200ms → 50ms

2. Redis 캐싱 추가
   - TTL: 300초
   - 캐시 히트율 목표: 70%
   - 예상 개선: 캐시 히트시 5ms

3. 쿼리 최적화
   - N+1 문제 해결 (JOIN으로 통합)
   - 예상 개선: 100ms → 20ms

중기 (1-2개월):
1. 데이터베이스 읽기 복제본 추가
   - Read-only 복제본 5개
   - Load balancing으로 부하 분산
```

```
2. GraphQL 도입
   - Over-fetching 방지
   - 필요한 필드만 쿼리

장기 (3개월):
1. Event-driven 아키텍처
   - 실시간 데이터 동기화
   - 이벤트 스트림 기반 업데이트

====================================
Step 4: 구현 계획 (HOW?)
====================================

Week 1: 데이터베이스 최적화
일정:
Mon-Tue: 인덱스 전략 수립 및 추가
  - EXPLAIN ANALYZE로 쿼리 분석
  - CREATE INDEX idx_user_status ON users(status, created_at)

Wed: Redis 캐싱 추가
  - Lua 스크립트로 원자성 보장
  - 캐시 무효화 전략 수립

Thu-Fri: 쿼리 최적화 (N+1 해결)
  - DataLoader 라이브러리 도입
  - 배치 쿼리 구현

테스트:
- Load 테스트: k6를 이용한 1만 RPS 테스트
- 캐시 히트율 모니터링
- 응답 시간 분포 분석

예상 결과:
- 응답 시간: 500ms → 120ms (76% 개선)

Week 2: 읽기 복제본 추가
- 복제 지연: < 100ms
- 장애 조치: 자동

Week 3: 모니터링 및 미세 조정

====================================
Step 5: 검증 계획
====================================

성공 기준:
✓ 평균 응답 시간 < 100ms
✓ p99 응답 시간 < 200ms
✓ 캐시 히트율 > 70%
✓ 에러율 < 0.01%

모니터링 메트릭:
- Prometheus + Grafana
- 리소스 사용률 (CPU, Memory)
- 데이터베이스 쿼리 시간 분포
- 캐시 히트/미스율

====================================
Step 6: 위험 관리
====================================
```

```
잠재적 위험:
1. 캐시 일관성 문제
   - 대책: Cache invalidation 전략 수립

2. 데이터베이스 마스터 부하 증가
   - 대책: 쓰기 배치 처리

3. 메모리 부족 (Redis)
   - 대책: LRU eviction 정책 설정

롤백 계획:
- 각 단계별 스냅샷 생성
- 문제 발견 시 이전 버전으로 즉시 복구
```

프로젝트 계획 통합 워크플로우

```
High-Level Project Planning Workflow:

1. /daily: 아침 회의
   "오늘의 일정과 우선순위"
   → 일일 계획 수립

2. /scope: 범위 정의
   "새로운 기능의 범위 명확히"
   → 요구사항 정의

3. /breakdown: 작업 분해
   "큰 작업을 작은 단위로"
   → User stories로 분해

4. /estimate: 기간 산정
   "각 작업의 예상 시간"
   → Story points 부여

5. /think + Sequential Thinking
   "복잡한 구현 전략 수립"
   → 상세 기술 계획

6. 개발 및 /review
   "코드 작성 및 리뷰"

7. /qa
   "품질 검사"

8. /ship
   "배포"

9. /retro
   "회고 및 학습"

This cycle repeats for continuous improvement.
```

5-8. 실전 통합 프로젝트 사례

사례: "AI 코드 리뷰 봇" SaaS 개발

프로젝트 규모: 2명 팀, 3개월 기간, $2,000 초기 MRR 목표

```
=====================================
Month 1: MVP 개발
=====================================

Week 1: CEO + Engineer 협업
1. /office-hours
   - 시장: GitHub 사용자 (개발자)
   - 차별점: LLM 기반 자동 코드 리뷰
   - GTM: Product Hunt 런칭

2. /plan-ceo-review
   - 비용 구조: OpenAI API 비용 고려
   - 가격 책정: 월 $29/$99/$299
   - KPI: 100명 가입 목표

3. /plan-eng-review
   - Backend: Python FastAPI
   - Frontend: Next.js
   - AI: GPT-4 API
   - 구현 기간: 4주

Week 2-3: 개발 집중
- Filesystem MCP: 프로젝트 구조 자동 생성
- /think: 복잡한 LLM 프롬프트 설계
  1️⃣ 코드 스타일 분석
  2️⃣ 버그 패턴 학습
  3️⃣ 성능 최적화 제안
  4️⃣ 보안 취약점 탐지

- GitHub MCP: 초기 커밋 및 자동화
- /review: 매일 코드 리뷰 (자동)

**GSTACK 워크플로우 시나리오 (MVP 개발 단계)**

Claude Code와의 실제 대화 흐름은 다음과 같습니다.

> "/think 로 LLM 코드 리뷰 프롬프트 전략을 설계해줘. 성능, 보안, 유지보수성, 테스트, 스타일 5가지 항목을 각각 점

Sequential Thinking이 프롬프트 구조, few-shot 예시, 출력 스키마를 단계별로 설계합니다.

> "설계된 프롬프트로 CodeReviewer 서비스를 FastAPI 라우트로 구현해줘. Filesystem MCP로 backend/app/services/revi

> "/review 스킬로 방금 생성한 reviewer.py 를 점검해줘."

> "GitHub MCP로 dev 브랜치에 커밋하고 PR 초안을 만들어줘."

핵심은 **코드를 직접 쓰지 않고 GSTACK 스킬과 MCP 조합에 위임**한다는 점입니다. 사용자는 요구사항 정의와 의사결정

Week 4: QA 및 배포 준비
- /qa: 테스트 커버리지 80% 확보
- /qa-only: 엣지 케이스 테스트
- /cso: 보안 검수 (API 키 관리, 데이터 암호화)

=====================================
Month 2: 베타 운영 및 개선
=====================================

Week 5-6: 초기 사용자 피드백
- Memory MCP: 피드백 저장 및 분석
- /daily: 우선순위 조정
```

```
- /think: 개선 전략 수립

주요 개선 사항:
1. 리뷰 정확도: 85% → 92%
2. 응답 시간: 8초 → 3초
3. 새 기능: GitHub Actions 통합

Week 7-8: 마케팅 준비
- /document-release: 변경사항 정리
- 블로그 포스트: "LLM으로 코드 리뷰 자동화"
- 데모 비디오 제작

====================================
Month 3: 상용화 및 확장
====================================

Week 9: Product Hunt 런칭
- /ship: 최종 배포 준비
- /canary: 트래픽 점진적 증대
  * Phase 1 (5%): 테스트 사용자
  * Phase 2 (25%): 초기 얼리 어댑터
  * Phase 3 (100%): 모든 사용자

주간 모니터링:
- 가입자: 50 → 150 → 400 (목표 500)
- 활성 사용자: 30% 유지율
- MRR: $500 → $1,500 → $2,200

Week 10-12: 팀 확장 및 로드맵
- /retro: 3개월 회고
  * 배운 점: LLM 프롬프트 엔지니어링의 중요성
  * 실수: 초기 인프라 스케일링 미비
  * 개선안: 자동 스케일링 구현

다음 분기 계획:
- 팀: 1명 엔지니어 추가
- 기능: GitLab, Bitbucket 지원
- 가격: 엔터프라이즈 플랜 추가
- 인프라: Kubernetes 마이그레이션

예상 12개월 MRR: $20,000
```

5-9. GSTACK 팁 및 베스트 프랙티스

효과적인 사용법

```
1. 올바른 스킬 선택
   ❌ /think로 단순 버그 수정 지시
   ✅ /think로 복잡한 알고리즘 설계 의뢰

   ❌ /review로 아이디어 브레인스토밍
   ✅ /review로 구현된 코드 평가

2. 입력 정보의 질이 출력 품질 결정
   ❌ "이 코드 리뷰해줘"
   ✅ "이 마이크로서비스의 N+1 쿼리를 해결하고 싶어.
        현재: 500ms, 목표: 100ms.
        스택: Node.js + PostgreSQL + Redis"

3. 반복적 개선
   /think → 계획 수립
   /plan-eng-review → 검증
   개발
   /review → 품질 확보
   /qa → 종합 검사

4. MCP 활용
   /think 후 GitHub MCP로 즉시 코드 작성
   Sequential Thinking 후 Filesystem MCP로 파일 생성
   계획 후 PostgreSQL MCP로 스키마 설계
```

Part 6: 취업 준비 및 면접 전략

> 금융권/공기업을 포함한 IT 취업을 위한 실전 준비 가이드입니다.

학습 목표

- 기술 면접 핵심 주제 정리
- 코딩 테스트 준비 전략
- 포트폴리오/이력서 최적화
- 금융/공기업 맞춤 취업 전략

6-1. 기술 면접 준비

yangshun/tech-interview-handbook (122K+ Stars)

링크: https://github.com/yangshun/tech-interview-handbook

개발자 면접을 위한 종합 가이드. 구성, 질문, 답변 전략까지 담고 있습니다.

📋 면접 유형별 전략

1. 코딩 면접 (Coding Interview)

인터뷰 전: 회사 기술 스택 조사, 최근 프로젝트 복습, 인터뷰어 LinkedIn 확인, 자주 묻는 문제 3-5개 준비
인터뷰 중: 질문 명확히 이해, 접근 방법 설명, 엣지 케이스 고려, 시간복잡도 분석, 트레이드오프 논의
코딩 후: 코드 리뷰, 테스트 케이스 작성, 성능 개선 제시, 결과 검증

2. 시스템 디자인 면접 프레임워크 (40분)

1. **요구사항 명확히 (5분)**: 함수형/비함수형, Scale, QPS, Data volume
2. **API 설계 (5분)**: Endpoints, Request/Response, Rate limiting
3. **데이터 모델 (5분)**: 스키마, 인덱싱, Sharding 전략
4. **High-level 설계 (10분)**: Components, Data flow, Cache, MQ

5. **상세 설계 (10분)**: 핵심 컴포넌트 심화, 병목 분석, 최적화
6. **마무리 (5분)**: Monitoring, Logging, Future improvements

실제 예: URL Shortener

- **요구사항**: 쓰기 500/s, 읽기 100,000/s, 가용성 99.9%, Latency < 200ms
- **용량 산정**: 5년 기준 약 78.8B URLs × 100B ≈ 7.88 TB
- **API**: `POST /api/shorten` (long_url → short_url), `GET /api/{code}` → 301 리다이렉트
- **데이터 모델**: `URLMapping(id PK, short_code UNIQUE, long_url, created_at, expiration)`
- **아키텍처**: Client → LB → API Servers → Redis Cache → MySQL, 분석은 Kafka로 비동기
- **핵심 기법**: Base62 인코딩, Collision은 retry+increment, LRU 캐시, Consistent Hashing

---

donnemartin/system-design-primer (290K+ Stars)

링크: https://github.com/donnemartin/system-design-primer

확장 가능한 시스템 설계를 위한 종합 리소스입니다.

🏗 핵심 개념

1. Scalability (확장성)

- **Vertical Scaling**: CPU/메모리 업그레이드. 간단하지만 단일 기계 한계 있음.
- **Horizontal Scaling**: 서버를 여러 대 추가하고 Load Balancer로 분산. 대부분 이쪽이 더 효율적.
- 전형적 구조: Client → LB → App1/2/3 → Redis Cache → DB (Master-Slave 복제)

2. 데이터베이스 설계

```
SQL vs NoSQL 선택

SQL (관계형 데이터베이스)
장점:
- ACID 보장
- 복잡한 쿼리 가능
- 데이터 무결성
단점:
- 수평 확장 어려움
- 고정된 스키마

예: MySQL, PostgreSQL

NoSQL (비관계형 데이터베이스)
장점:
- 수평 확장 용이
- 빠른 읽기/쓰기
- 유연한 스키마
단점:
- ACID 보장 안 함
- 쿼리 능력 제한
- Consistency 문제

예: MongoDB, Cassandra, DynamoDB
```

3. 캐싱 전략

- LRU, LFU, Write-Through, Write-Back, Cache-Aside 다섯 가지 전략이 빈출 주제입니다.
- 면접 팁: 전략을 고를 때 "읽기/쓰기 비율", "정합성 요구", "히트율" 세 기준으로 설명하세요.
- Python `functools.lru_cache`, `collections.OrderedDict`로 직접 구현해본 경험을 면접에서 얘기할 수 있으면 강점.

---

kdn251/interviews (63K+ Stars)

링크: https://github.com/kdn251/interviews

Java 기반 면접 준비용 알고리즘 및 자료구조 구현.

🎯 Java 기반 면접 준비

- 이 레포는 LeetCode 빈출 문제의 Java 풀이를 Array/String/Tree/DP 등 카테고리별로 정리합니다.
- 추천 학습법: 문제를 직접 풀어본 뒤 레포 풀이와 비교, 시간/공간 복잡도를 꼭 따져보세요.
- 대기업 면접 빈출: Two Sum, Valid Palindrome, Reverse Integer, Merge Intervals, LRU Cache.

> Claude Code 활용 팁
> `> "kdn251/interviews 레포의 Dynamic Programming 섹션에서 금융권 면접 빈출 Top 5를 골라서 시간복잡도와 핵심 아이디어를 비교해줘"`

6-2. 금융/공기업 취업에 도움되는 레포지토리

B-1. 금융 IT 관련 레포지토리

1) QuantConnect/Lean - 알고리즘 트레이딩 엔진 (10K+ Stars)

레포 개요

- QuantConnect가 개발한 오픈소스 알고리즘 트레이딩 엔진
- C#과 Python을 지원하는 백테스팅 및 실시간 거래 플랫폼
- 금융권 퀀트/개발자 직무 면접에서 자주 언급되는 프로젝트

설치 방법

```
# Python을 이용한 기본 설치
git clone https://github.com/QuantConnect/Lean.git
cd Lean

# C# 환경 필요 (Linux/Mac)
dotnet --version  # .NET 5.0 이상

# Docker를 이용한 설치 (권장)
docker pull quantconnect/lean:latest
docker run -it quantconnect/lean:latest
```

기본 사용법 - Python API

```python
from AlgorithmImports import *

class BasicTradingAlgorithm(QCAlgorithm):
    def Initialize(self):
        # 1년의 데이터로 시뮬레이션
        self.SetStartDate(2023, 1, 1)
        self.SetEndDate(2024, 1, 1)

        # 초기 자본금
        self.SetCash(100000)

        # 거래할 주식 추가
        self.spy = self.AddEquity("SPY", Resolution.Daily).Symbol
        self.bond = self.AddEquity("AGG", Resolution.Daily).Symbol

    def OnData(self, data):
        # 보유 자산이 없으면 60/40 포트폴리오 구성
        if not self.Portfolio.Invested:
            self.SetHoldings(self.spy, 0.6)
            self.SetHoldings(self.bond, 0.4)

        # 간단한 매매 신호 (SMA 활용)
        if not self.spy in data:
            return

        # 거래 로직 추가 가능
        price = data[self.spy].Close
        self.Debug(f"SPY Price: {price}")
```

실전 활용 시나리오: 이동평균 교차 전략 (Moving Average Crossover)

```python
from AlgorithmImports import *

class MovingAverageCrossover(QCAlgorithm):
    def Initialize(self):
        self.SetStartDate(2020, 1, 1)
        self.SetEndDate(2024, 1, 1)
        self.SetCash(100000)

        self.spy = self.AddEquity("SPY", Resolution.Daily).Symbol

        # 단기 이동평균(50일)과 장기 이동평균(200일)
        self.fast_ma = self.SMA(self.spy, 50, Resolution.Daily)
        self.slow_ma = self.SMA(self.spy, 200, Resolution.Daily)

    def OnData(self, data):
        if not self.fast_ma.IsReady or not self.slow_ma.IsReady:
            return

        fast = self.fast_ma.Current.Value
        slow = self.slow_ma.Current.Value

        # Golden Cross: 단기 MA가 장기 MA를 위로 뚫고 올라감
        if fast > slow and not self.Portfolio[self.spy].Invested:
            self.SetHoldings(self.spy, 1.0)
            self.Debug(f"Golden Cross! Buy at {data[self.spy].Close}")

        # Death Cross: 단기 MA가 장기 MA를 아래로 뚫고 내려감
        elif fast < slow and self.Portfolio[self.spy].Invested:
            self.Liquidate(self.spy)
            self.Debug(f"Death Cross! Sell at {data[self.spy].Close}")
```

금융권 면접 활용 팁

- Lean을 이용한 백테스팅 경험을 포트폴리오에 추가하면 강점
- 면접에서 "자신의 트레이딩 전략을 구현해본 경험"을 증명 가능
- 코드 최적화, 위험 관리(Risk Management) 등 금융 개념을 함께 설명

관련 레포 추천

- `QuantConnect/Documentation` - 공식 문서
- `ranaroussi/yfinance` - Yahoo Finance 데이터 수집
- `quantopian/*` - 구 Quantopian 자산들

---

2) OpenBB-finance/OpenBB - 투자 분석 플랫폼 (35K+ Stars)

레포 개요

- Bloomberg 터미널을 대체하는 오픈소스 투자 분석 플랫폼
- 주식, 암호화폐, 옵션 등 다양한 자산 분석 도구 제공
- 데이터 분석 + API 설계를 배우기에 최적

설치 방법

```
# pip를 이용한 설치
pip install openbb

# 또는 소스코드에서 설치
git clone https://github.com/OpenBB-finance/OpenBB.git
cd OpenBB
pip install -e .

# 설치 확인
python -c "import openbb; print(openbb.__version__)"
```

기본 사용법 - 주식 데이터 조회

```
import openbb

# 개별 주식 가격 데이터
apple_data = openbb.stocks.get_historical_data("AAPL", start_date="2023-01-01")
print(apple_data.head())

# 주식 정보 조회
info = openbb.stocks.get_company_info("AAPL")
print(f"Apple Market Cap: {info['marketCap']}")

# 재무제표 조회
income_stmt = openbb.stocks.get_income_statement("AAPL", period="annual")
print(income_stmt)

# 기술적 분석 지표
rsi = openbb.stocks.indicators.relative_strength_index("AAPL", period=14)
print(f"AAPL RSI(14): {rsi}")
```

실전 활용 시나리오: 다중 종목 비교 분석

```python
import openbb
import pandas as pd

# 기술주 3개 종목 비교
stocks = ["AAPL", "GOOGL", "MSFT"]
comparison_data = {}

for stock in stocks:
    # 최근 1년 데이터
    data = openbb.stocks.get_historical_data(stock, start_date="2023-01-01")

    # 수익률 계산
    returns = data['Adj Close'].pct_change().dropna()

    # 통계 정보
    comparison_data[stock] = {
        "평균 수익률": returns.mean() * 252,  # 연간화
        "변동성": returns.std() * (252 ** 0.5),
        "Sharpe Ratio": (returns.mean() / returns.std()) * (252 ** 0.5),
        "최대 낙폭": (data['Adj Close'] / data['Adj Close'].cummax() - 1).min()
    }

# DataFrame으로 변환하여 비교
comparison_df = pd.DataFrame(comparison_data).T
print(comparison_df)

# 금융 지표 비교
print("\n=== 금융 지표 비교 ===")
for stock in stocks:
    pe_ratio = openbb.stocks.get_pe_ratio(stock)
    pb_ratio = openbb.stocks.get_pb_ratio(stock)
    print(f"{stock}: P/E = {pe_ratio:.2f}, P/B = {pb_ratio:.2f}")
```

금융권 면접 활용 팁

- OpenBB API 설계를 분석하면 좋은 학습 자료
- 금융 데이터 파이프라인 구축 경험을 어필 가능
- "데이터 기반 투자 의사결정 시스템" 포트폴리오 프로젝트 제작 권장

관련 레포 추천

- `OpenBB-finance/OpenBBTerminal` - CLI 버전
- `OpenBB-finance/docs` - 공식 문서

B-2. 공기업 코딩테스트 준비

1) TheAlgorithms/Python - 알고리즘 구현 (195K+ Stars)

레포 개요

- Python으로 구현된 모든 알고리즘 모음 (정렬, 탐색, 동적계획법 등)
- NCS(국가직무능력표준) 및 공기업 코딩테스트 대비에 최적
- 각 알고리즘의 시간복잡도와 설명이 상세함

설치 방법

```
git clone https://github.com/TheAlgorithms/Python.git
cd Python

# 폴더 구조 확인
ls -la
# output:
# sorts/          - 정렬 알고리즘
# searches/       - 탐색 알고리즘
# dynamic_programming/  - 동적계획법
# graphs/         - 그래프 알고리즘
# arrays/         - 배열 관련
# strings/        - 문자열 처리
```

공기업 코딩테스트 추천 학습 순서

```
Week 1: 정렬 & 탐색
├─ searches/linear_search.py (선형 탐색)
├─ searches/binary_search.py (이진 탐색)
└─ sorts/bubble_sort.py, merge_sort.py, quick_sort.py

Week 2: 자료구조
├─ arrays/ (배열 조작)
├─ linked_lists/ (연결 리스트)
└─ stacks_and_queues/ (스택/큐)

Week 3: 동적계획법
├─ dynamic_programming/fibonacci.py
├─ dynamic_programming/longest_increasing_subsequence.py
└─ dynamic_programming/knapsack.py

Week 4: 그래프
├─ graphs/breadth_first_search.py
├─ graphs/depth_first_search.py
└─ graphs/dijkstra.py (최단경로)

Week 5-8: 실전 문제
├─ 프로그래머스 Level 2-3 (공기업 기출)
├─ Baekjoon 실버~골드 (NCS 난이도)
└─ LeetCode Medium 풀이 검증
```

기본 코드 예제: 이진 탐색

```python
# TheAlgorithms/Python의 binary_search.py 분석

def binary_search(arr, target):
    """
    정렬된 배열에서 target을 찾는 이진 탐색
    시간복잡도: O(log n)
    공간복잡도: O(1)
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1  # 오른쪽 절반 탐색
        else:
            right = mid - 1  # 왼쪽 절반 탐색

    return -1  # 찾지 못함

# 테스트
sorted_array = [1, 3, 5, 7, 9, 11, 13, 15]
print(binary_search(sorted_array, 7))  # 출력: 3
print(binary_search(sorted_array, 6))  # 출력: -1
```

공기업 실전 문제: 최소공배수 (LCM) - NCS 유형

```python
from math import gcd

def lcm(a, b):
    """최소공배수 계산"""
    return (a * b) // gcd(a, b)

def lcm_multiple(numbers):
    """여러 수의 최소공배수"""
    result = numbers[0]
    for num in numbers[1:]:
        result = lcm(result, num)
    return result

# 예제: 공기업 코딩테스트 기출
# 문제: 3, 5, 7의 최소공배수를 구하시오
print(lcm_multiple([3, 5, 7]))  # 출력: 105
```

동적계획법 예제: 피보나치 수열

```python
def fibonacci_memo(n, memo={}):
    """메모이제이션을 이용한 피보나치"""
    if n in memo:
        return memo[n]
    if n <= 1:
        return n

    memo[n] = fibonacci_memo(n-1, memo) + fibonacci_memo(n-2, memo)
    return memo[n]

def fibonacci_dp(n):
    """DP 테이블을 이용한 피보나치 (더 빠름)"""
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[0], dp[1] = 0, 1

    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]

# 비교
print(f"메모이제이션: {fibonacci_memo(30)}")  # 832040
print(f"DP: {fibonacci_dp(30)}")  # 832040 (훨씬 빠름)
```

관련 레포 추천

- `neetcode-io/leetcode` - LeetCode 풀이 (다음 섹션)
- `Baekjoon-Online-Judge/` - 백준 풀이 참고

---

2) neetcode-io/leetcode - LeetCode 풀이 및 코딩 테스트 가이드 (6K+ Stars)

레포 개요

- LeetCode 상위 150문제의 최적 풀이 모음
- 공기업 코딩테스트의 난이도와 유사한 문제들 포함
- 각 문제마다 풀이 설명 및 시간/공간복잡도 분석

설치 및 사용

```
git clone https://github.com/neetcode-io/leetcode.git
cd leetcode

# 구조 확인
ls -la
# 주요 폴더:
# 1-array-hashing/
# 2-two-pointers/
# 3-sliding-window/
# 4-stack/
# 5-binary-search/
# 6-linked-list/
# 7-trees/
# 8-tries/
# 9-heap-priority-queue/
# 10-graphs/
# 11-advanced-graphs/
# 12-1-d-dp/
# 13-2-d-dp/
# 14-greedy/
# 15-intervals/
# 16-math-geometry/
```

공기업 코딩테스트 추천 우선순위

```
높음 (반드시 풀어야 할 주제):
├ Array & Hashing (배열, 해시맵)
├ Two Pointers (투 포인터)
├ Sliding Window (슬라이딩 윈도우)
└ Binary Search (이진 탐색)

중간 (자주 출제):
├ Stack & Queue (스택, 큐)
├ Linked List (연결 리스트)
├ Tree & Graph (트리, 그래프)
└ Dynamic Programming (동적계획법)

낮음 (심화):
├ Greedy (탐욕법)
├ Math & Geometry (수학, 기하)
└ Trie (트라이)
```

Array & Hashing - Two Sum (LeetCode #1)

```python
def twoSum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        if target - num in seen:
            return [seen[target - num], i]
        seen[num] = i
    return []
```

핵심 아이디어: "이전에 본 수"를 해시맵에 저장해 O(n)에 해결. 3Sum/4Sum으로 확장될 때는 정렬 후 투 포인터($O(n^2)$)가 정석.

Sliding Window - Longest Substring Without Repeating Characters

```python
def lengthOfLongestSubstring(s):
    """
    반복되지 않는 가장 긴 부분 문자열의 길이
    슬라이딩 윈도우 활용 (O(n) 풀이)
    """
    char_index = {}
    max_length = 0
    start = 0

    for end, char in enumerate(s):
        # 문자가 현재 윈도우 내에 이미 존재하면
        if char in char_index and char_index[char] >= start:
            start = char_index[char] + 1

        char_index[char] = end
        max_length = max(max_length, end - start + 1)

    return max_length

# 테스트
print(lengthOfLongestSubstring("abcabcbb"))  # 출력: 3 ("abc")
print(lengthOfLongestSubstring("bbbbb"))      # 출력: 1 ("b")
print(lengthOfLongestSubstring("pwwkew"))     # 출력: 3 ("wke")
```

Binary Search - Search in Rotated Sorted Array

```python
def search(nums, target):
    """
    회전된 정렬 배열에서 target 찾기
    예: [4,5,6,7,0,1,2] 에서 0 찾기
    """
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid

        # 왼쪽 절반이 정렬되어 있는 경우
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        # 오른쪽 절반이 정렬되어 있는 경우
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1

# 테스트
print(search([4, 5, 6, 7, 0, 1, 2], 0))  # 출력: 4
print(search([4, 5, 6, 7, 0, 1, 2], 3))  # 출력: -1
```

학습 전략: 일일 1문제 풀이 계획

```
Day 1-5: Array & Hashing (5문제)
Day 6-10: Two Pointers (5문제)
Day 11-15: Sliding Window (5문제)
Day 16-20: Stack & Queue (5문제)
Day 21-25: Binary Search (5문제)
Day 26-35: DP (10문제) - 공기업 필출 유형
Day 36-45: Tree & Graph (10문제)
Day 46-50: 기출 문제 풀이 (5문제)

총 50일 × 1문제 = 2-3개월 완성
```

관련 레포 추천

- `TheAlgorithms/Python` - 알고리즘 구현 (이전 섹션)
- `Baekjoon-Online-Judge/` - 백준 기출 풀이

B-3. 보안 & 인프라 (금융권 필수)

1) OWASP/CheatSheetSeries - 보안 체크시트 (28K+ Stars)

레포 개요

- OWASP(Open Web Application Security Project)의 공식 보안 체크시트
- 금융권, 공기업에서 필수적인 보안 규정 및 구현 가이드
- Web, API, Cloud 등 분야별 보안 베스트 프랙티스

설치 및 접근

```
git clone https://github.com/OWASP/CheatSheetSeries.git
cd CheatSheetSeries

# 주요 체크시트 확인
ls -la cheatsheets/

# 예: Authentication Cheat Sheet
cat cheatsheets/Authentication_Cheat_Sheet.md

# 또는 웹브라우저로 접근
# https://cheatsheetseries.owasp.org/
```

금융권 필수 보안 항목

```
1. Authentication Security (인증 보안)
   ├─ Password Storage Cheat Sheet
   ├─ Session Management Cheat Sheet
   └─ Multi-Factor Authentication (MFA) 구현

2. Cryptography (암호화)
   ├─ Cryptographic Storage Cheat Sheet
   ├─ HTTPS Implementation
   └─ Key Management

3. API Security (API 보안)
   ├─ REST Security Cheat Sheet
   ├─ GraphQL Cheat Sheet
   └─ OAuth 2.0 Implementation

4. Data Protection (데이터 보호)
   ├─ PCI DSS 준수
   ├─ GDPR 대응
   └─ 개인정보 암호화
```

실전 예제: 안전한 비밀번호 저장 (Password Hashing)

```python
# OWASP 권장: bcrypt 사용 (절대 평문 저장 금지!)

import bcrypt
from passlib.context import CryptContext

# bcrypt를 이용한 비밀번호 저장
def hash_password(password: str) -> str:
    """
    비밀번호를 안전하게 해시화
    OWASP: bcrypt with salt (최소 12라운드)
    """
    # salt는 자동으로 생성됨
    hashed = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))
    return hashed.decode('utf-8')

def verify_password(password: str, hashed: str) -> bool:
    """해시된 비밀번호 검증"""
    return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))

# 사용 예제
original_password = "MySecurePassword123!"
hashed_password = hash_password(original_password)

print(f"해시된 비밀번호: {hashed_password}")
print(f"검증 성공: {verify_password(original_password, hashed_password)}")
print(f"검증 실패: {verify_password('WrongPassword', hashed_password)}")
```

실전 예제: SQL Injection 방지 (준비된 명령문 사용)

```python
# OWASP: Parameterized Queries 사용
import sqlite3

# 잘못된 방법 (SQL Injection 취약점!)
def unsafe_query(username):
    query = f"SELECT * FROM users WHERE username = '{username}'"
    # 문제: username에 ' OR '1'='1 같은 악의적 입력 가능

# 올바른 방법 (OWASP 권장)
def safe_query(username):
    conn = sqlite3.connect(':memory:')
    cursor = conn.cursor()

    # 파라미터화된 쿼리 사용 (?)
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    return cursor.fetchall()

# 사용 예제
safe_query("admin' OR '1'='1")  # 안전함 (문자열로 처리됨)
```

실전 예제: HTTPS 및 SSL/TLS 구성

```python
# Flask를 이용한 안전한 HTTPS 설정
from flask import Flask
from flask_talisman import Talisman

app = Flask(__name__)

# OWASP: 보안 헤더 설정
Talisman(app,
    force_https=True,  # HTTP를 HTTPS로 강제 리다이렉트
    strict_transport_security=True,  # HSTS 활성화
    content_security_policy={
        'default-src': "'self'",
        'script-src': "'self' 'unsafe-inline'",
        'style-src': "'self' 'unsafe-inline'"
    }
)

@app.route('/')
def hello():
    return 'Secure Connection'

if __name__ == '__main__':
    # SSL/TLS 인증서로 실행
    app.run(ssl_context='adhoc')  # 또는 'cert.pem', 'key.pem'
```

금융권 면접 질문 대비

```
Q1: SQL Injection이란? 방어 방법?
A: 악의적 SQL 코드를 입력하는 공격.
   파라미터화된 쿼리(Prepared Statements) 사용으로 방어

Q2: XSS(Cross-Site Scripting)와 방어?
A: 사용자 입력에 악의적 스크립트를 삽입하는 공격.
   입력 검증, 출력 인코딩, CSP(Content Security Policy) 사용

Q3: OWASP Top 10이란?
A: 웹 애플리케이션의 가장 위험한 10가지 보안 취약점
   (Injection, Broken Authentication, Sensitive Data Exposure 등)

Q4: 금융권에서 필수인 보안 표준?
A: PCI DSS (결제카드 산업 데이터 보안 표준)
   - 비암호화 민감 데이터 저장 금지
   - 강력한 액세스 제어
   - 정기적 보안 감사
```

관련 레포 추천

- `OWASP/OWASP-Top-10-2021` - OWASP Top 10 최신 버전
- `ircrazierii/owasp-top-10-labs` - 실습 랩

---

2) trimstray/the-book-of-secret-knowledge - 시스템 관리자/DevOps 완전 가이드 (160K+ Stars)

레포 개요

- Linux, 네트워크, 보안, DevOps 관련 심화 지식 모음
- 금융권 인프라 엔지니어 필수 학습 자료
- 실무에서 자주 사용되는 명령어, 설정, 베스트 프랙티스

설치 및 접근

```
git clone https://github.com/trimstray/the-book-of-secret-knowledge.git
cd the-book-of-secret-knowledge

# README.md가 전체 내용의 목차
# 웹브라우저로도 접근 가능
# https://github.com/trimstray/the-book-of-secret-knowledge
```

금융권 인프라 필수 주제

```
1. Linux Administration (리눅스 관리)
   ├─ 파일 시스템 권한 설정 (chmod, chown)
   ├─ 프로세스 관리 (ps, kill, systemd)
   ├─ 네트워크 설정 (ip, netstat, ss)
   └─ 로그 관리 (journalctl, /var/log)

2. Security (보안)
   ├─ SSH 설정 및 키 관리
   ├─ 방화벽 설정 (iptables, firewalld)
   ├─ 접근 제어 (ACL, SELinux)
   └─ 감시 및 모니터링

3. Network (네트워크)
   ├─ TCP/IP 기초
   ├─ DNS 설정
   ├─ VPN 및 TLS/SSL
   └─ 로드 밸런싱

4. DevOps (데브옵스)
   ├─ Docker 컨테이너화
   ├─ 자동화 배포
   ├─ CI/CD 파이프라인
   └─ 모니터링 및 로깅
```

실전 명령어: 보안 감시

```
# 1. 열린 포트 확인 (nmap 사용)
nmap -sV localhost
# 결과: 어떤 서비스가 어떤 포트에서 실행 중인지 확인

# 2. 네트워크 연결 확인 (ss 사용, netstat은 deprecated)
ss -tlnp
# t: TCP, l: Listening, n: Numeric, p: Process

# 3. 파일 무결성 확인 (md5sum)
md5sum /path/to/critical/file
# 금융권: 중요 파일의 해시값을 정기적으로 검증

# 4. 방화벽 규칙 확인 (firewalld)
sudo firewall-cmd --list-all
sudo firewall-cmd --add-port=8080/tcp --permanent
sudo firewall-cmd --reload

# 5. 시스템 보안 감사 (lynis)
sudo apt install lynis
sudo lynis audit system
```

실전 명령어: 성능 모니터링

```
# 1. CPU 사용률 모니터링
top -b -n 1 | head -20
# b: batch mode, n: 실행 횟수

# 2. 메모리 사용률 확인
free -h
# 금융 거래: 메모리 누수는 심각한 장애

# 3. 디스크 사용률 확인
df -h
du -sh /var/log/*  # 로그 파일 크기 확인

# 4. 네트워크 대역폭 모니터링
iftop -n  # 실시간 네트워크 모니터링

# 5. 프로세스별 리소스 사용 확인
ps aux --sort=-%mem | head -10  # 메모리 사용량 순
ps aux --sort=-%cpu | head -10  # CPU 사용량 순
```

실전 예제: SSH 보안 설정

```
# SSH 설정 파일 보안화 (/etc/ssh/sshd_config)

# 금융권 필수 설정:
# 1. 루트 로그인 비활성화
sed -i 's/#PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config

# 2. 비밀번호 인증 비활성화 (키 기반 인증만 사용)
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config

# 3. SSH 포트 변경 (기본 22 제외)
sed -i 's/#Port 22/Port 2222/' /etc/ssh/sshd_config

# 4. 설정 검증
sshd -t

# 5. SSH 서비스 재시작
sudo systemctl restart sshd

# 6. SSH 키 쌍 생성 (클라이언트)
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
# 결과: ~/.ssh/id_rsa (개인 키), ~/.ssh/id_rsa.pub (공개 키)

# 7. 공개 키를 서버에 배포
ssh-copy-id -i ~/.ssh/id_rsa.pub -p 2222 user@server.com

# 8. SSH 접속 (비밀번호 입력 없음)
ssh -i ~/.ssh/id_rsa -p 2222 user@server.com
```

실전 예제: 시스템 로그 모니터링

```
# 금융권에서 필수: 모든 접근 및 시스템 변경 기록

# 1. journalctl을 이용한 로그 조회
journalctl --unit=sshd -n 20  # SSH 최근 20개 로그
journalctl -u docker -n 20     # Docker 컨테이너 로그

# 2. 특정 시간대 로그 조회
journalctl -u sshd --since "2024-01-01" --until "2024-01-02"

# 3. 실시간 로그 모니터링
journalctl -f  # tail -f와 유사

# 4. 로그 필터링 (특정 사용자의 실패한 로그인)
grep "Failed password" /var/log/auth.log | grep "user@ip"

# 5. 로그 분석: 비정상 접속 탐지
journalctl -u sshd | grep "Failed\|Accepted" | tail -50
```

금융권 인프라 면접 대비

```
Q1: Linux 권한 설정 (chmod)의 의미?
A: chmod 755 = rwx r-x r-x (소유자:읽기+쓰기+실행, 그룹:읽기+실행, 기타:읽기+실행)
   금융권: 민감한 파일은 600 (소유자만 접근)

Q2: 방화벽 설정 필요 이유?
A: 불필요한 포트 차단으로 공격 표면 최소화
   금융권: 화이트리스트 방식 (필요한 포트만 개방)

Q3: SSH 키 기반 인증의 장점?
A: 비밀번호 탈취 불가, Brute-force 공격 방어
   금융권: 필수 요구사항

Q4: 로그 모니터링이 중요한 이유?
A: 침입 탐지, 감시 증적, 규제 준수
   금융권: 감시자 로그(audit log) 유지 의무
```

관련 레포 추천

- `OWASP/CheatSheetSeries` (이전 섹션)
- `awesome-selfhosted/awesome-selfhosted` - 자체 호스팅 가이드

B-4. DevOps & CI/CD

1) kubernetes/kubernetes - 컨테이너 오케스트레이션 (113K+ Stars)

레포 개요

- 구글에서 개발한 오픈소스 컨테이너 오케스트레이션 플랫폼
- 마이크로서비스 아키텍처의 표준 인프라
- 금융권 대규모 시스템에서 필수적인 기술

설치 및 기본 설정

```bash
# 1. kubectl 설치 (Kubernetes CLI)
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# 2. kubectl 버전 확인
kubectl version --client

# 3. 로컬 테스트용 Minikube 설치
curl -minikube https://github.com/kubernetes/minikube/releases/download/latest/minikube-linux-amd64
chmod +x minikube-linux-amd64
sudo mv minikube-linux-amd64 /usr/local/bin/minikube

# 4. Minikube 시작
minikube start
minikube status

# 5. Kubernetes 클러스터 확인
kubectl cluster-info
kubectl get nodes
```

기본 개념: Pod, Service, Deployment

```
# 1. Pod: Kubernetes의 최소 배포 단위 (Docker 컨테이너 래퍼)
# pod.yaml 파일 작성
cat > pod.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
EOF

# Pod 생성
kubectl apply -f pod.yaml

# Pod 확인
kubectl get pods
kubectl describe pod nginx-pod

# Pod 로그 확인
kubectl logs nginx-pod
```

실전 예제: Deployment 핵심 필드

`deployment.yaml`의 필수 요소는 다음과 같습니다.

- `apiVersion: apps/v1`, `kind: Deployment`
- `spec.replicas: 3` (Pod 복제본)
- `spec.selector.matchLabels`와 `spec.template.metadata.labels` 일치
- `containers[].image`, `containers[].ports.containerPort`
- `resources.requests/limits` (memory/cpu)
- `livenessProbe.httpGet` (헬스체크)

배포: `kubectl apply -f deployment.yaml`, 상태 확인: `kubectl get deployments/pods`, 업데이트: `kubectl set image deployment/web-app web=nginx:1.21`, 롤백: `kubectl rollout undo deployment/web-app`.

실전 예제: Service / ConfigMap / Secret

- **Service** (`type: LoadBalancer`): `selector.app`으로 Pod를 선택, `port` → `targetPort` 매핑
- **ConfigMap**: 환경 변수 주입용 (예: `DATABASE_HOST`, `LOG_LEVEL`)
- **Secret** (`type: Opaque`): 민감 정보는 base64 인코딩 — 금융권에선 Vault/KMS 연동 필수

금융권 Kubernetes 필수 설정

- **Namespace**: 팀/환경별 격리 (`kind: Namespace`, `metadata.name: finance`)

- **NetworkPolicy**: 기본 deny-all 후 화이트리스트. `policyTypes: [Ingress, Egress]`, `podSelector: {}`
- **RBAC**: `Role`(권한 정의) + `RoleBinding`(사용자-권한 연결). 예: `verbs: [get, list, watch]`만 허용

> Claude Code 활용 팁
> `> "내 Spring Boot 앱(8080 포트)을 K8s에 배포하는 deployment.yaml + service.yaml을 만들고, 금융권 기준 NetworkPolicy와 RBAC도 같이 정의해줘"`

관련 레포 추천
- `kubernetes/kubernetes` - 공식 저장소
- `kelseyhightower/kubernetes-the-hard-way` - Kubernetes 완전 학습

---

2) actions/runner - GitHub Actions 실행 환경 (5K+ Stars)

레포 개요
- GitHub Actions의 공식 실행 환경 (runner)
- CI/CD 파이프라인 구축의 핵심
- 금융권에서 자동화 테스트 및 배포에 필수

GitHub Actions 기본 개념

```
# GitHub 저장소에서 .github/workflows 디렉토리 생성
mkdir -p .github/workflows

# 워크플로우 파일 작성
cat > .github/workflows/ci.yml << 'EOF'
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.9'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Run tests
      run: |
        pytest tests/ -v --cov=src

    - name: Security scan
      run: |
        bandit -r src/ -f json -o bandit-report.json

    - name: Upload coverage
      uses: codecov/codecov-action@v3
      with:
        file: ./coverage.xml
EOF
```

실전 예제: Python 프로젝트 CI/CD 파이프라인

Python 앱의 전형적인 워크플로우는 3개 job으로 구성됩니다.

- **test job**: `actions/checkout` → `setup-python` (matrix로 3.8-3.11) → `pip install` → `flake8`, `mypy`, `pytest --cov` → `codecov-action`으로 커버리지 업로드
- **security job**: `bandit`, `semgrep --config=p/security-audit`, `safety check`
- **deploy job**: `needs: [test, security]`에 `if: github.ref == 'refs/heads/main'` 조건을 걸어 스테이징 배포

금융권 필수 CI/CD 설정

금융권은 위 기본 파이프라인에 아래 단계를 추가로 요구합니다.

- 코드 품질 게이트: `pylint --fail-under=9.0`, `black --check`, `isort --check-only`
- 의존성 취약점: `pip-audit`, `safety check`
- 암호화/컴플라이언스 자체 스크립트 (`verify-encryption.sh`, `check-compliance.sh`)
- 부하 테스트: `locust --headless -u 100 -r 10 --run-time 60s`
- SBOM 생성: `cyclonedx-bom -o sbom.xml`
- approval-gate: PR 승인 2인 이상 체크 (`actions/github-script`로 `pulls.listReviews` 검증)

> Claude Code 활용 팁
>
> `> "금융권 CI/CD 요구사항(코드 품질/보안/SBOM/2인 승인)을 모두 충족하는 GitHub Actions workflow를 내 Python 프로젝트에 맞게 만들어줘"`

GitHub Actions 사용 예제: 자동 버전 관리

```yaml
# .github/workflows/release.yml
name: Automated Release

on:
  push:
    tags:
      - 'v*'

jobs:
  release:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Create Release Notes
      run: |
        # 커밋 로그 기반 릴리스 노트 생성
        git log $(git describe --tags --abbrev=0)..HEAD --oneline > RELEASE_NOTES.md

    - name: Create GitHub Release
      uses: actions/create-release@v1
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      with:
        tag_name: ${{ github.ref }}
        release_name: Release ${{ github.ref }}
        body_path: RELEASE_NOTES.md
```

학습 전략: GitHub Actions 마스터

```
Step 1: 기본 워크플로우 작성 (1주)
├─ Checkout, Setup Python
├─ Install dependencies
└─ Run tests

Step 2: CI/CD 자동화 (1주)
├─ Code quality checks
├─ Security scans
└─ Automated testing

Step 3: 배포 자동화 (1주)
├─ Docker build & push
├─ Kubernetes deployment
└─ Health checks

Step 4: 고급 기능 (1주)
├─ Matrix builds (다양한 환경)
├─ Conditional steps
└─ Secrets 관리

총 4주 학습
```

관련 레포 추천

- `actions/runner` - 공식 저장소
- `awesome-actions/awesome-actions` - GitHub Actions 라이브러리 모음

부록

부록 A: Claude Code Plan 모드 완전 가이드

A-1. Plan 모드란?

Plan 모드는 Claude Code의 **변경사항을 미리 보여준 후** 적용하는 안전한 작업 방식입니다.

일반 모드 vs Plan 모드

```
┌─ 일반 모드 (Direct Mode)
│  └─ 명령 실행 → 즉시 결과 반영
│     장점: 빠름
│     단점: 실수 수정 어려움
│
└─ Plan 모드 (Preview Mode)
   └─ 계획 작성 → 미리 보기 → 승인 후 실행
      장점: 안전, 검토 가능
      단점: 약간의 추가 시간
```

A-2. Plan 모드 활성화 방법

방법 1: 키보드 단축키

```
1. Shift + Tab 을 두 번 연속으로 누름
2. 또는 Shift + Tab + Tab (한 번에)
3. Claude Code 인터페이스의 "Plan Mode" 토글 활성화
```

방법 2: 명령어 사용

```
메시지 입력창에서 다음 중 하나 입력:
- /plan (또는 /plan-mode)
- "이것을 Plan 모드로 실행해줘"
- "미리 보기를 보여줘"

예: "/plan 이 파일의 코드를 리팩토링해주세요"
```

방법 3: 메뉴 버튼

```
Claude Code 우측 상단 메뉴 → Plan Mode 선택
```

A-3. Plan 모드 vs Direct 모드 사용 시기

Plan 모드 추천:

```
✓ 프로젝트 구조 변경 (파일 생성/이동/삭제)
✓ 기존 코드 대폭 수정
✓ 설정 파일 변경 (database.yml, config.json 등)
✓ 중요한 파일 편집
✓ 팀 프로젝트 작업
✓ 프로덕션 코드 변경
```

Direct 모드 추천:

```
✓ 간단한 오류 수정
✓ 주석 추가
✓ 문서 작성
✓ 로그 파일 검토
✓ 빠른 테스트
✓ 개인 프로젝트 소규모 작업
```

A-4. Plan 모드 단계별 워크플로우

예제: React 컴포넌트 리팩토링

Step 1: Plan 모드 활성화

사용자가 `/plan React 컴포넌트를 함수형으로 리팩토링해줘`라고 입력하면, Claude가 다음과 같은 계획을 보여줍니다.

- 클래스형 → 함수형 변환
- Hooks 적용 (useState, useEffect)
- 라이프사이클 메서드 제거
- PropTypes 유지

Step 2: 미리보기 확인

변경 전 (클래스형):

```
class UserProfile extends React.Component {
  constructor(props) {
    super(props);
    this.state = { user: null };
  }
  componentDidMount() {
    fetchUser(this.props.userId).then(user => this.setState({ user }));
  }
  render() {
    return <div>{this.state.user?.name}</div>;
  }
}
```

변경 후 (함수형 + Hooks):

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    fetchUser(userId).then(setUser);
  }, [userId]);
  return <div>{user?.name}</div>;
}
```

Step 3: 변경 사항 승인

버튼 옵션: `Apply Changes` / `Edit Plan` / `Discard`. "Apply Changes"를 클릭합니다.

Step 4: 최종 적용

- 컴포넌트 리팩토링 완료
- 파일: `src/components/UserProfile.jsx`
- 변경 라인: 8 → 13 (5줄 단축)

A-5. Plan 편집 (Ctrl+G)

계획을 생성한 후 내용을 조정할 수 있습니다.

```
Plan 생성 후:
┌──────────────────────────────────
│ 📋 PLAN: 함수 추상화
│ ├─ 1. getUser() 함수 추출
│ ├─ 2. formatUserData() 함수 추출
│ └─ 3. 테스트 코드 추가
│
│ [🔧 Ctrl+G로 편집] [ ✓ 적용 ] [ ✗ 취소 ]
└──────────────────────────────────

편집 모드:
┌──────────────────────────────────
│ 1. getUser() 함수 추출 ✓
│ 2. formatUserData() 함수 추출 ✓
│ 3. 테스트 코드 추가 ✓
│ 4. (새로 추가) JSDoc 주석 추가
│
│ [완료]
└──────────────────────────────────
```

A-6. Plan 모드 베스트 프랙티스

좋은 Plan 모드 사용:

```
# ✓ 좋은 예: 구체적인 지시
"Plan 모드로 다음을 해주세요:
1. User 클래스의 validate_email() 메서드를 unittest로 테스트
2. 테스트 케이스: 유효한 이메일, 잘못된 형식, None 값
3. 테스트 파일: tests/test_user.py에 추가"

# ✗ 나쁜 예: 너무 모호함
"코드 개선해줘"
```

Plan 모드 검토 체크리스트:

```
변경 사항 미리보기 확인 시:
☑ 변경된 파일 목록 확인
☑ 삭제되는 코드 없는지 확인 (의도하지 않은 삭제)
☑ 신규 코드의 로직 검증
☑ 기존 테스트 영향도 확인
☑ 설정 파일 변경이 있으면 특히 주의

의심스러운 부분:
☑ "?" 또는 "..." 표시된 부분 재확인
☑ 대량 삭제 작업은 특히 주의
☑ 데이터베이스 스키마 변경은 double-check
```

부록 B: Sequential Thinking 완전 가이드

B-1. Sequential Thinking MCP란?

Sequential Thinking은 **단계별 논리적 추론**을 통해 복잡한 문제를 해결하는 MCP(Message Collection Protocol) 도구입니다.

특징:

```
일반 응답:
Q: "100을 소인수분해 해줄래?"
A: 100 = 2² × 5² (즉시 답변)

Sequential Thinking:
Q: "100을 소인수분해 해줄래?"
→ Step 1: 100을 2로 나누면? 50
→ Step 2: 50을 2로 나누면? 25
→ Step 3: 25를 5로 나누면? 5
→ Step 4: 5를 5로 나누면? 1
→ 결론: 2² × 5²

장점: 논리 검증 가능, 실수 수정 용이
```

B-2. Sequential Thinking 설치

설치 명령어:

```bash
# Claude Code CLI에서 설치
claude-code install sequential-thinking-mcp

# 또는 수동 설치
npm install -g sequential-thinking-mcp

# 설치 확인
claude-code mcp list | grep sequential
```

Claude Code 설정 파일 (`claude.json`):

```json
{
  "mcps": {
    "sequential-thinking": {
      "enabled": true,
      "timeout": 30000
    }
  }
}
```

B-3. Sequential Thinking 작동 원리

다단계 추론 프로세스:

```
입력: 복잡한 문제
  ↓
[Step 1] 문제 분해
  - 주요 요소 파악
  - 선행 조건 확인
  ↓
[Step 2] 단계별 분석
  - 각 단계의 근거 제시
  - 중간 결과 검증
  ↓
[Step 3] 오류 확인
  - 논리적 모순 탐지
  - 가정 재검토
  ↓
[Step 4] 결론 도출
  - 최종 답변 생성
  - 신뢰도 제시
  ↓
출력: 검증된 답변
```

B-4. Sequential Thinking 사용 시기

추천 시나리오:

```
✓ 알고리즘 설계 및 분석
  예: "N-Queen 문제의 백트래킹 알고리즘을 설계해주세요"

✓ 아키텍처 결정
  예: "마이크로서비스 vs 모놀리식 아키텍처 비교"

✓ 버그 분석
  예: "메모리 누수 원인을 단계별로 찾아주세요"

✓ 복잡한 요구사항 분석
  예: "금융 거래 시스템의 보안 요구사항 정리"

✓ 성능 최적화
  예: "데이터베이스 쿼리 최적화 전략"

✗ 부추천 시나리오:
  - 간단한 질문 ("Python 문법 확인")
  - 빠른 답변 필요 (실시간 채팅)
  - 이미 검증된 일반 지식
```

B-5. Sequential Thinking 실제 예제

예제 1: 복잡한 알고리즘 설계

```
사용자 요청:
"Sequential Thinking으로 분석해줄래?
 1000개의 정렬된 배열에서 target을 찾는
 가장 효율적인 방법을 설계해줘"

Sequential Thinking 프로세스:
═══════════════════════════════

[THINKING Step 1: 문제 분석]
- 입력: 정렬된 배열 (크기 1000), target 값
- 제약: 효율성 중심
- 가능한 방법:
  1. 선형 탐색: O(n)
  2. 이진 탐색: O(log n) ← 가장 효율적
  3. 해시 테이블: O(1) but 전처리 필요

[THINKING Step 2: 이진 탐색 상세 분석]
- 시간복잡도: O(log 1000) ≈ 10번 비교
- 공간복잡도: O(1)
- 구현: 재귀 vs 반복
  └ 반복이 스택 오버플로우 방지

[THINKING Step 3: 코드 설계]
```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```
```

[THINKING Step 4: 검증]
- 최악의 경우: $\log_2(1000)$ = ~10회 → 가장 효율적
- 엣지 케이스:
- target이 배열에 없음: -1 반환 ✓
- target이 처음/마지막: 정상 동작 ✓

[결론]
이진 탐색이 최적 (O(log n) 시간복잡도)

```
**예제 2: 금융 거래 시스템 설계**
```

사용자 요청:
"Sequential Thinking으로 설계해줄래?
금융 거래 플랫폼에서
'동시에 2개 거래가 같은 계좌에서 발생하면?'
이 문제를 단계별로 분석해줘"

Sequential Thinking 프로세스:
━━

[THINKING Step 1: 문제 정의]
- 시나리오: 사용자 계좌 잔액 $1000
- 거래 A: $600 출금 (동시)
- 거래 B: $600 출금 (동시)
- 문제:
- Race condition (동시성 문제)
- 둘 다 성공하면 -$200 (불가능!)

[THINKING Step 2: 원인 분석]
Database 레벨에서 문제:

```
시간  거래 A              거래 B
t1    select balance=1000 -
t2    -                   select balance=1000
t3    update balance=400  -
t4    -                   update balance=400
결과: 최종 balance = $400 (오류!)
```

[THINKING Step 3: 해결 방안 3가지]

방안 1: Database Lock (비관적 잠금)

```sql
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE id=1 FOR UPDATE;
-- 이 행을 다른 트랜잭션이 접근 불가
UPDATE accounts SET balance = balance - 600 WHERE id=1;
COMMIT;
```

장점: 간단, 안전
단점: 성능 저하 (다른 거래 대기)

방안 2: MVCC (다중 버전 동시성 제어)

```sql
-- PostgreSQL, MySQL InnoDB
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
SELECT balance FROM accounts WHERE id=1;
UPDATE accounts SET balance = balance - 600 WHERE id=1;
COMMIT; -- 충돌 감지 시 자동 롤백
```

장점: 높은 동시성
단점: 복잡, 롤백 처리 필요

방안 3: Optimistic Locking (낙관적 잠금)

```
-- version 컬럼 사용
SELECT balance, version FROM accounts WHERE id=1;
UPDATE accounts
  SET balance = balance - 600, version = version + 1
  WHERE id=1 AND version = 1;
-- 실패 시 재시도
```

[THINKING Step 4: 금융권 규정 검토]
- PCI DSS: 트랜잭션 원자성 필수
- 은행법: ACID 보장 의무
 └── Atomicity (원자성): 모두 성공 또는 모두 실패

[결론]
금융권에서는 "Database Lock" 또는 "MVCC"로
ACID 보장 필수. 성능은 차선책.

```
### B-6. Sequential Thinking과 GSTACK 통합

**GSTACK 워크플로우 + Sequential Thinking:**
```

프로젝트 계획 단계:
1. /office-hours
 └── "Sequential Thinking으로 분석해줄래?
우리 프로젝트의 아키텍처 선택지"

1. Sequential Thinking으로 비교:
   ├── 마이크로서비스 (장단점 분석)
   ├── 모놀리식 (장단점 분석)
   └── 하이브리드 (절충안)

2. /plan-ceo-review
   └── 위 분석 결과를 비즈니스 관점에서 평가

3. 최종 결정
   └── 근거 기반 아키텍처 선택 완료

```
---
## 부록 C: 추천 학습 로드맵

### C-1. 비전공자 → 풀스택 개발자 (12개월 로드맵)

**전제 조건:**
- 기본 컴퓨터 활용 능력
- 영어 기술 문서 읽기 능력 (번역기 사용 가능)
- 주 20-30시간 학습 시간

**Month 1-2: 웹 기초 (4주, 80시간)**
```

학습 목표: HTML/CSS/JavaScript 기초 완성

Week 1-2: HTML & CSS
├── freeCodeCamp "Responsive Web Design" (300시간 → 40시간 집중)
├── HTML5 시맨틱 요소
├── CSS Grid & Flexbox
└── 프로젝트: 포트폴리오 웹사이트 (반응형)

Week 3-4: JavaScript 기초
├── 변수, 자료형, 연산자
├── 조건문, 반복문, 함수
├── DOM 조작
└── 프로젝트: 계산기, Todo 리스트

추천 자료:
- freeCodeCamp JavaScript (4시간 강의)
- MDN Web Docs (레퍼런스)
- Codepen (코드 샘플)

```
**Month 3-4: React 기초 (4주, 80시간)**
```

학습 목표: React 컴포넌트 기반 개발

Week 1-2: React 기초
├── JSX 문법
├── 컴포넌트 (함수형)
├── Props와 State
├── Hooks (useState, useEffect, useContext)
└── 프로젝트: 날씨 앱, 영화 검색

Week 3-4: React 심화
├── 커스텀 Hooks
├── 성능 최적화 (React.memo, useMemo)
├── 라우팅 (React Router)
└── 프로젝트: SNS 피드, 블로그

추천 자료:
- React Official Tutorial
- Dave Ceddia "Pure React"
- Scrimba React Course
- 프로젝트: GitHub API를 이용한 사용자 검색

```
**Month 5-6: Spring Boot 백엔드 (4주, 80시간)**
```

학습 목표: REST API 개발 및 데이터베이스 연동

Week 1-2: Spring Boot 기초
├── 프로젝트 구조 (MVC 패턴)
├── Controller, Service, Repository
├── JPA/Hibernate ORM
├── 데이터베이스 연동 (MySQL)
└── 프로젝트: 간단한 CRUD API

Week 3-4: Spring Boot 심화
├── Spring Security (인증/인가)
├── JWT 토큰 인증
├── 에러 처리
├── 로깅 및 모니터링
└── 프로젝트: 사용자 관리 시스템

추천 자료:
- Spring Boot Official Guide
- Baeldung (튜토리얼 사이트)
- Java Spring Framework 강의 (인프런/Udemy)
- 프로젝트: 블로그 API (CRUD + 댓글 기능)

```
**Month 7-8: 데이터베이스 & API (4주, 80시간)**
```

학습 목표: 데이터베이스 설계 및 고급 API 개발

Week 1-2: 데이터베이스
├── SQL 기본 (SELECT, INSERT, UPDATE, DELETE)
├── JOIN, GROUP BY, 서브쿼리
├── 인덱싱 및 최적화
├── 정규화 (1NF, 2NF, 3NF)
└── 실습: Leetcode Database 문제 (Medium Level)

Week 3-4: 고급 API
├── 페이지네이션
├── 필터링 및 정렬
├── 캐싱 (Redis)
├── API 문서화 (Swagger/OpenAPI)
└── 프로젝트: 전자상거래 API (상품 필터링, 주문 관리)

추천 자료:
- SQL Tutorial (W3Schools)
- Database Design Basics (Stanford)
- Redis 공식 문서
- 프로젝트: 금융 데이터 API (시계열 데이터)

```
**Month 9-10: 실전 프로젝트 & 포트폴리오 (4주, 80시간)**
```

학습 목표: 풀스택 포트폴리오 프로젝트 완성

Week 1: 요구사항 분석 & 설계
├── 프로젝트 선정 (비즈니스 가치 있는 주제)
├── API 설계
├── 데이터베이스 스키마 설계
└── UI/UX 디자인

Week 2-3: 개발
├── 백엔드 API 개발
├── 프론트엔드 구현
├── 테스트 작성 (Jest, JUnit)
└── 배포 준비

Week 4: 완성 & 배포
├── 문서 작성 (README, API 문서)
├── 배포 (Heroku, AWS, Vercel)
├── CI/CD 설정 (GitHub Actions)
└── 포트폴리오 정리

추천 프로젝트 아이디어:
1. 금융 관리 앱 (예산 관리, 수입/지출 추적)
2. 스터디 그룹 매칭 플랫폼
3. 부동산 검색 시스템
4. 온라인 쇼핑몰
5. 실시간 협업 문서 편집기

```
**Month 11-12: 면접 준비 & 취업 (4주)**
```

Week 1-2: 알고리즘 & 코딩테스트
├── LeetCode Easy → Medium (50문제)
├── 자료구조 (Array, String, Tree, Graph, DP)
└── 모의 면접

Week 3: 기술 면접 준비
├── 시스템 디자인 (마이크로서비스, 캐싱, 데이터베이스)
├── 포트폴리오 프로젝트 설명 연습
├── 기술 질문 준비
└── 행동 면접 (Behavioral Interview)

Week 4: 취업 활동
├── 이력서 작성
├── 포트폴리오 정리

├── 채용공고 지원 (매일 3-5개)
└── 면접 스케줄링

추천 자료:
- Leetcode + 알고리즘 강의
- "시스템 디자인 면접" 책
- Cracking the Coding Interview

```
---

### C-2. 금융권 취업 준비 로드맵 (6개월 가속화 과정)

**전제 조건:**
- 기본 프로그래밍 능력 (다른 언어 경험 있음)
- 금융권 취업 목표

**Month 1: Java & Spring Boot 집중 (4주)**
```

주 목표: 금융권 표준 스택 습득

Week 1: Java 심화
├── 객체지향 프로그래밍 (OOP)
├── 컬렉션 프레임워크 (List, Map, Set)
├── 멀티스레딩 (Thread, Synchronization)
└── 람다식 및 스트림 API

Week 2-3: Spring Boot 실전
├── Dependency Injection
├── Spring Data JPA
├── Spring Security (OAuth2, JWT)
├── 트랜잭션 관리 (ACID)
└── 프로젝트: 계좌 관리 시스템

Week 4: 금융 도메인 지식
├── 주식 거래 프로토콜
├── 결제 시스템 (PCI DSS)
├── 암호화 통신
└── 금융 API (OpenBanking)

프로젝트: 은행 계좌 관리 API
(이체, 거래 내역, 계좌 잔액)

```
**Month 2: SQL & 데이터베이스 (4주)**
```

주 목표: 금융권 데이터 처리 능력

Week 1-2: SQL 마스터
├── 복잡한 쿼리 최적화
├── 윈도우 함수 (ROW_NUMBER, RANK)
├── 트리거 및 스토어드 프로시저
├── 인덱스 전략
└── 실습: LeetCode Database 문제 (Medium)

Week 3: 금융 데이터 모델
├── 계좌/거래 테이블 설계
├── 시계열 데이터 처리
├── 감사 로그 (Audit Log)
└── 데이터 정규화

Week 4: 성능 튜닝
├── 쿼리 실행 계획 분석
├── 인덱스 최적화
├── 캐싱 전략 (Redis)
└── 프로젝트: 금융 데이터 분석 쿼리

프로젝트: "사용자별 월간 거래 통계" 쿼리 최적화
(복합 인덱싱, 파티셔닝)

```
**Month 3: 코딩테스트 (4주)**
```

주 목표: 공기업/대기업 채용 대비

Week 1-2: 알고리즘 (60문제)
├── 동적계획법 (DP) - 15문제
├── 그래프 (BFS, DFS) - 15문제
├── 정렬 및 탐색 - 15문제
├── 문자열 처리 - 15문제
└── 실습: Baekjoon Gold Level

Week 3: NCS 유형 코딩테스트
├── "한국은행" 기출 (구글 검색)
├── "공기업" 기출 문제풀이
├── 제한 시간 내 풀이 (60분/1문제)
└── 프로젝트: 모의고사 5회

Week 4: 모의 면접
├── 알고리즘 설명하기 (화이트보드)
├── 코드 리뷰 (시간복잡도, 공간복잡도)
├── 최적화 과정 설명
└── 예상 질문 대비

추천 플랫폼:
- LeetCode (영문, 상세 설명)
- Baekjoon (한국 온라인저지, 기출)
- Programmers (카카오/네이버 기출)

```
**Month 4: 금융 & 보안 (4주)**
```

주 목표: 금융권 보안 및 규정 이해

Week 1: 금융 IT 기초
├── 금융감독 규정 (ISMS, PCI DSS)
├── 암호화 통신 (TLS/SSL)
├── 토큰 기반 인증 (JWT, OAuth2)
└── 프로젝트: 안전한 로그인 시스템

Week 2: 보안 검사
├── OWASP Top 10 분석
├── SQL Injection 방어
├── XSS, CSRF 방어
└── 보안 코드 리뷰

Week 3: 금융 도메인 심화
├── 이중 인증 (2FA)
├── 거래 한도 관리
├── 부정 거래 탐지
└── PCI DSS 준수

Week 4: 면접 대비
├── "금융권에서 중요한 보안은?"
├── "암호화 알고리즘 선택 기준"
├── "거래 시스템 설계"
└── 프로젝트: 거래 검증 시스템

프로젝트: "거래 이중 검증 시스템"
(OTP, 전자서명, 부정 거래 탐지)

```
**Month 5: DevOps & 인프라 (4주)**
```

주 목표: 금융권 운영 능력

Week 1: Docker & Kubernetes
├── 컨테이너 기초 (Docker)
├── 오케스트레이션 (Kubernetes)
├── 배포 자동화
└── 프로젝트: Spring Boot 앱 컨테이너화

Week 2: CI/CD 파이프라인
├── GitHub Actions
├── 자동 테스트
├── 자동 배포
└── 프로젝트: 풀스택 자동 배포

Week 3: 모니터링 & 로깅
├── ELK 스택 (Elasticsearch, Logstash, Kibana)
├── 성능 모니터링 (Prometheus)
├── 로그 분석
└── 금융권 감시 로그 수집

Week 4: 고가용성 설계
├── 로드 밸런싱
├── 데이터베이스 복제 (Replication)
├── 재해 복구 (DR) 계획
└── 프로젝트: 고가용성 시스템 설계

추천 학습:
- Kubernetes Official Tutorial
- GitHub Actions Documentation
- ELK Stack 공식 문서

```
**Month 6: 포트폴리오 & 면접 (4주)**
```

Week 1-2: 금융 프로젝트 완성
├── 프로젝트: 종합 금융 관리 플랫폼
│ ├── 사용자 계좌 관리
│ ├── 주식 거래 시뮬레이션
│ ├── 거래 내역 분석
│ └── 이중 인증 및 보안
├── 배포 (AWS/Azure)
├── 성능 최적화 (응답시간 < 200ms)
└── 문서화 (API 명세, 아키텍처)

Week 3: 기술 면접 준비
├── 시스템 디자인 (금융 시스템)
├── 성능 최적화 사례
├── 보안 사례
├── 운영 경험 설명
└── 모의 면접 3회

Week 4: 취업 활동
├── 이력서/자소서 작성
├── 포트폴리오 발표 준비

├── 금융사 채용공고 지원
└── 최종 면접 스케줄링

```
**금융권 취업 최종 체크리스트:**
```

기술 스킬:
☑ Java/Spring Boot (심화)
☑ SQL (최적화까지)
☑ 알고리즘 (Gold Level)
☑ 보안 (OWASP Top 10)
☑ DevOps (Docker, K8s)

포트폴리오:
☑ 금융 주제 프로젝트 (완성도 높음)
☑ GitHub README 상세 작성
☑ 배포 가능한 상태
☑ 성능/보안 최적화 증명

면접 준비:
☑ 알고리즘 설명 능력
☑ 시스템 디자인 경험
☑ 금융 도메인 지식
☑ 팀 협업 경험 설명
☑ 자신만의 기술 인사이트

지원 전략:
☑ 대형 금융사 (은행, 증권, 보험)
☑ 핀테크 (토스, 뱅샐, 당근페이)
☑ 공기업 (우정사업본부, 교보 등)
☑ 면접일정 집중 공략

```
---
## 부록 D: 유용한 리소스 링크 모음

#### D-1. 한국 개발자 커뮤니티

**블로그 플랫폼:**
```

1. Velog (https://velog.io/)
   - 개발자 블로그 플랫폼
   - 추천: 기술 글 품질 높음
   - 팔로우: "해시코드", "코딩알림", "노드버드"

2. Tistory (https://www.tistory.com/)
   - 개인 기술 블로그
   - 추천 블로그: "기억보다는 기록을", "코딩하는 생각"

3. Medium 한국 (https://medium.com/)
   - 국제 기술 글
   - 추천: "Better Programming", "Towards Data Science"

4. dev.to (https://dev.to/)
   - 개발자 커뮤니티
   - 영문이지만 예제 풍부

```
**개발자 커뮤니티:**
```

1. GitHub 한국 (https://github.com/korean-developers)
   - 한국 개발자 네트워크
   - 채용 정보, 프로젝트 협업

2. 프로그래머스 (https://programmers.co.kr/)
   - 코딩테스트 + 채용
   - 월간 코딩 챌린지

3. 백준 온라인 저지 (https://www.acmicpc.net/)
   - 알고리즘 문제 풀이
   - 한국 수준 높음

4. 인프런 (https://www.inflearn.com/)
   - 온라인 코딩 강의
   - 가성비 최고 (한국)

5. 노마드 코더 (https://nomadcoders.co/)
   - 웹 개발 강의
   - YouTube 채널 추천

```
### D-2. 온라인 저지 (Online Judge)

**알고리즘 연습:**
```

1. LeetCode (https://leetcode.com/)
   주소: 영문
   특징: 상세 분석, 상위 풀이 학습
   추천: 비전공자는 Easy → Medium
   료: 유료 구독 권장 ($35/월)

2. Baekjoon (https://www.acmicpc.net/)
   주소: 한국
   특징: 한국 대학 경시대회 기출
   추천: 공기업/대기업 코딩테스트 대비
   료: 무료

3. Programmers (https://programmers.co.kr/learn/challenges)
   특징: 카카오/네이버 기출

   추천: 취업 준비생 필수
   료: 무료

4. Codeforces (https://codeforces.com/)
   특징: 경쟁 프로그래밍
   추천: 고급 알고리즘
   료: 무료

학습 전략:
Week 1-4: LeetCode Easy (50문제)
Week 5-8: LeetCode Medium (50문제)
Week 9-12: Baekjoon Silver (50문제)
Week 13-16: Baekjoon Gold (50문제)
총 3-4개월에 200문제

```
**시뮬레이션 환경:**
```

1. HackerRank (https://www.hackerrank.com/)
   - 회사 채용 문제
   - C, Java, Python 지원

2. TopCoder (https://www.topcoder.com/)
   - 알고리즘 경시대회
   - 상금 대회

3. AtCoder (https://atcoder.jp/)
   - 일본 알고리즘 대회
   - 한국 수준보다 높음

```
### D-3. 무료 온라인 강좌

**기본부터 심화까지:**
```

1. freeCodeCamp (https://www.freecodecamp.org/)
   - 완전 무료
   - 영상 강좌 (최고 품질)
   추천 코스:
   ├── Responsive Web Design (5시간)
   ├── JavaScript Algorithms (4시간)
   ├── React (12시간)
   └── Backend Development (12시간)

2. Khan Academy (https://www.khanacademy.org/)
   - 컴퓨터 과학 기초
   - 영상 + 문제 풀이

3. Codecademy (https://www.codecademy.com/)
   - 인터랙티브 학습
   - 유료 구독 권장

4. Udemy (https://www.udemy.com/)
   - 저가 강의 ($10-15)
   - 추천: "The Complete JavaScript Course"
   "The Complete Python Course"

한국 강의:
5. 인프런 (https://www.inflearn.com/)
- 한국 강사, 한국어
- 가격: 15,000원 (세일 시 3,000원)
추천: "부트캠프" 시리즈

1. 웹개발 종합반 (https://spartacodingclub.kr/)
   - 입문자 친화적
   - 무료 + 유료 모두 제공

```
**전문 기술 학습:**
```

1. Docker (https://www.docker.com/101-tutorial/)
   - 공식 튜토리얼 무료

2. Kubernetes (https://kubernetes.io/docs/tutorials/)
   - 공식 문서, 완전 무료

3. Spring Boot (https://spring.io/guides)
   - 공식 가이드 무료

4. React 공식 (https://react.dev/)
   - 최신 React 학습
   - 완전히 리뉴얼됨 (2022)

```
### D-4. 도서 추천

**필독서:**
```

1. "Cracking the Coding Interview"
   - 저자: Gayle Laakmann McDowell
   - 대상: 취업 준비 필수
   - 가격: $40 (한국어 번역본: 45,000원)

2. "System Design Interview"
   - 저자: Alex Xu
   - 대상: 시니어 준비
   - 가격: $50 (한국어 번역본: 50,000원)

3. "Clean Code"
   - 저자: Robert C. Martin
   - 대상: 코드 품질 개선
   - 가격: $45 (한국어: 40,000원)

4. "Effective Java"
   - 저자: Joshua Bloch
   - 대상: Java 심화
   - 가격: $55 (한국어: 45,000원)

5. "데이터베이스 설계와 구현"
   - 저자: 정정수
   - 대상: 한국 저자, SQL 완벽 학습
   - 가격: 35,000원

```
**알고리즘 & 자료구조:**
```

1. "알고리즘 문제 풀이 핸드북"
   - 저자: 하용호
   - 한국 고등학생이 쓴 저자
   - 가격: 35,000원

2. "코딩 인터뷰 완전 분석"
   - 저자: 한빛 미디어
   - LeetCode 스타일 문제
   - 가격: 32,000원

```
**금융 & DevOps:**
```

1. "금융 시스템 설계"
   - 저자: 구글 엔지니어 (강추)
   - 가격: Udemy $40

2. "쿠버네티스 완벽 가이드"
   - 저자: Kelsey Hightower
   - 무료: "Kubernetes The Hard Way"
   - 가격: 온라인 무료

3. "마이크로서비스 아키텍처"
   - 저자: Sam Newman
   - 가격: 35,000원

```
### D-5. 유튜브 채널 추천

**기초부터 심화까지:**
```

영어 채널:
1. freeCodeCamp (https://www.youtube.com/@freeCodeCamp)
- 완전 무료, 고품질
- 인기: "100 Days of Code", "Web Development"

1. Traversy Media (https://www.youtube.com/@TraversyMedia)
   - 프론트엔드 심화
   - 인기: "React Crash Course", "Node.js Tutorial"

2. The Net Ninja (https://www.youtube.com/@NetNinja)
   - 웹 개발 완전 입문
   - 재미있고 명확한 설명

3. CodeWithHarry (https://www.youtube.com/@CodeWithHarry)
   - Python 입문 최고
   - 100개 이상 영상

한국 채널:
5. 노마드 코더 (https://www.youtube.com/@nomadcoders)
- 웹 개발 전문
- 한국인, 한국어

1. 코딩애플 (https://www.youtube.com/@CodingApple)
   - HTML/CSS/JS 입문
   - 영상 퀄리티 높음

2. 우디 (https://www.youtube.com/@woodywood6)
   - Spring Boot, Java
   - 실무급 심화 강의

3. 동빈나 (https://www.youtube.com/@dongbinna)
   - 알고리즘 및 경제
   - 한국 최고의 알고리즘 채널

```
**라이브 코딩 & 면접:**
```

1. Google Developers (https://www.youtube.com/@GoogleDevelopers)
   - Google 엔지니어 강의
   - 시스템 디자인 고급

2. Back to Back SWE (https://www.youtube.com/@BackToBackSWE)
   - 면접 완벽 준비
   - 실무 경험 공유

3. Fireship (https://www.youtube.com/@Fireship)
   - 기술 트렌드 설명
   - 10분 완성 영상 (추천!)

```
### D-6. 채용 정보 플랫폼

**대기업/공기업:**
```

1. 사람인 (https://www.saramin.co.kr/)
   - 대형 포탈
   - 필터: 개발자 > 신입

2. 잡코리아 (https://www.jobkorea.co.kr/)
   - 공기업 주로 게재
   - 필터: 공공기관

3. 워크넷 (https://www.work.go.kr/)
   - 정부 공식 플랫폼
   - 공기업/공공기관 집중

4. 공기업아이 (https://www.gonggigeop.ai/)
   - 공기업 전문 플랫폼
   - 추천 많음

```
**스타트업/핀테크:**
```

1. 원티드 (https://www.wanted.co.kr/)
   - 스타트업 채용 전문
   - 필터: "개발자 경력 신입"

2. 프로그래머스 채용 (https://programmers.co.kr/jobs)
   - 코딩테스트 + 채용
   - 통합 플랫폼

3. 정글 (https://jungle.co.kr/)
   - 초기 스타트업
   - 보상: 스톡옵션 + 급여

4. 앤젤리스트 (https://wellfound.com/)
   - 세계 스타트업
   - 한국 시작업도 있음

```
**금융권 채용:**
```

1. 금융감독원 (https://www.fss.or.kr/)
   - 공식 채용 정보
   - "IT 연봉 높음" 검색어

2. 각 은행 사이트
   - 신한은행 (https://www.shinhan.com/shbank/recruit)

- 국민은행 (https://www.kbbank.com/recruit)
- 우리은행 (https://www.wooribank.com/)

3. 핀테크 채용
   - 토스 (https://toss.im/career)
   - 당근 (https://about.daangn.com/jobs/)
   - 뱅샐 (https://banksalad.com/careers)

```
### D-7. 학습 효율을 위한 Tip

**최고의 학습 방법:**
```

Tip 1: 80/20 원칙
- 20%의 핵심 개념으로 80%의 문제 해결
- 예: React hooks만으로 90% 프로젝트 가능
- 시작: 기초만 완벽하게 (심화는 나중)

Tip 2: 프로젝트 기반 학습
- 책만 읽기 vs 프로젝트 하면서 배우기
- 70% 프로젝트 경험이 필수
- 추천: 3-4개 포트폴리오 프로젝트

Tip 3: 반복학습 (Spaced Repetition)
- 배운 것을 다음날, 1주 후, 1개월 후 복습
- 앱: Anki (플래시카드)
- 추천: 알고리즘 패턴 외우기

Tip 4: 설명하기 (Teaching Others)
- 배운 내용을 블로그로 쓰기
- 남에게 설명하기
- 면접처럼 화이트보드에 설명해보기

Tip 5: 코드 리뷰 받기
- GitHub에 올리고 피드백 받기
- 오픈소스 PR 보내기
- 멘토에게 검토받기

Tip 6: 습관화
- 하루 3시간 × 3개월 > 일주일 30시간
- 매일 정해진 시간에 학습
- 추천: 아침 1시간 + 저녁 2시간

```
**학습 일정 관리:**
```

월요일: 새로운 개념 학습
화-목요일: 프로젝트 구현

금요일: 복습 + 코드 리뷰
토-일요일: 개인 프로젝트 진행

주간 목표:
- 강의 3시간
- 프로젝트 10시간
- 알고리즘 7시간
- 복습 및 글쓰기 5시간
총 25시간/주

```
---

---

## 부록 E: 가이드북 빌드 파이프라인 사용법

이 가이드북은 `scripts/` 디렉토리의 Python 스크립트로 자동 생성됩니다.

### 빌드 방법

```bash
# 의존성 설치
pip install -r scripts/requirements.txt

# 전체 빌드 (MD + DOCX + PDF)
python scripts/build_guidebook.py

# 특정 포맷만 빌드
python scripts/build_guidebook.py --format md
python scripts/build_guidebook.py --format docx
python scripts/build_guidebook.py --format pdf
```
```

파이프라인 구조

```
src/*.md → md_merger.py → output/Guidebook.md
                                ↓
                      ┌──────────┴──────────┐
                      ↓                     ↓
               md_to_docx.py        md_to_pdf.py
                      ↓                     ↓
            output/Guidebook.docx  output/Guidebook.pdf
```

사용된 라이브러리

라이브러리	용도	Part 4 참조
python-docx	MD → DOCX 변환	4-1
WeasyPrint	HTML → PDF 렌더링	4-4 (확장)
markdown	MD → HTML 변환	-
Pygments	코드 구문 강조	-